



Πανεπιστήμιο Δυτικής Αττικής
Σχολή Μηχανικών
Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Διπλωματική Εργασία

Μελέτη και ανάπτυξη τεχνικών για την παρακολούθηση και την
αποσφαλμάτωση της εκτέλεσης εντολών σε υπολογιστικά
συστήματα

Χρήστος Μαργιώλης
Α.Μ. 19390133

Εισηγητής: Παναγιώτης Καρκαζής, Αν. Καθηγητής

Διπλωματική Εργασία

Μελέτη και ανάπτυξη τεχνικών για την παρακολούθηση και την αποσφαλμάτωση της εκτέλεσης εντολών σε υπολογιστικά συστήματα

Χρήστος Μαργιώλης
Α.Μ. 19390133

Εισηγητής:

Παναγιώτης Καρκαζής, Αν. Καθηγητής

Εξεταστική επιτροπή:

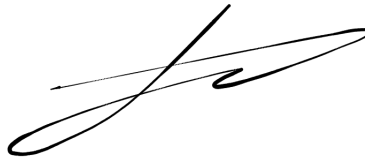
Παναγιώτης Καρκαζής, Αν. Καθηγητής
Νικόλαος Μυριδάκης, Αν. Καθηγητής
Κωνσταντίνος Μαυρομμάτης, Λέκτορας
Ημερομηνία εξέτασης: 15 Οκτωβρίου 2025

Δήλωση συγγραφέα διπλωματικής εργασίας

Ο κάτωθι υπογεγραμμένος **Μαργιώλης Χρήστος** του **Δημητρίου**, με αριθμό μητρώου **19390133**, φοιτητής του Τμήματος Μηχανικών Πληροφορικής και Υπολογιστών της Σχολής Μηχανικών του Πανεπιστημίου Δυτικής Αττικής, δηλώνω υπεύθυνα ότι:

«Βεβαιώνω ότι είμαι συγγραφέας αυτής της Διπλωματικής εργασίας και κάθε βοήθεια την οποία είχα για την προετοιμασία της, είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, οι όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε ακριβώς είτε παραφρασμένες, αναφέρονται στο σύνολό τους, με πλήρη αναφορά στους συγγραφείς, τον εκδοτικό οίκο ή το περιοδικό, συμπεριλαμβανομένων και των πηγών που ενδεχομένως χρησιμοποιήθηκαν από το διαδίκτυο. Επίσης, βεβαιώνω ότι αυτή η εργασία έχει συγγραφεί από μένα αποκλειστικά και αποτελεί προϊόν πνευματικής ιδιοκτησίας τόσο δικής μου, όσο και του Ιδρύματος. Παράβαση της ανωτέρω ακαδημαϊκής μου ευθύνης αποτελεί ουσιώδη λόγο για την ανάκληση του πτυχίου μου».

Ο Δηλών



Ευχαριστίες

Την προσπάθειά μου αυτή υποστήριξε ο επιβλέπων καθηγητής μου Παναγιώτης Καρκαζής, τον οποίο θα ήθελα να ευχαριστήσω.

Ακόμα θα ήθελα να ευχαριστήσω την οικογένειά μου για τη συμπαράστασή τους κατά τη διάρκεια των σπουδών μου, καθώς και τους Mark Johnston, Mitchell Horne και το FreeBSD Foundation, για την πολύτιμη βοήθειά τους.

Περίληψη

Η εργασία αποσκοπεί στην μελέτη τεχνικών που χρησιμοποιούνται στην ανάλυση και αποσφαλμάτωση λογισμικού μέσω της καταγραφής και παρακολούθησης των εντολών που εκτελούνται σε έναν επεξεργαστή. Στο πλαίσιο της διπλωματικής εργασίας θα σχεδιαστεί και θα αναπτυχθεί επέκταση του εργαλείου DTrace η οποία θα παρέχει την δυνατότητα παρακολούθησης οποιασδήποτε μεμονωμένης εντολής assembly εντός μιας δεδομένης συνάρτησης του πυρήνα του λειτουργικού συστήματος FreeBSD.

Λέξεις κλειδιά: FreeBSD, DTrace, Παρακολούθηση, Αποσφαλμάτωση, Λειτουργικά συστήματα, C, Assembly, Αρχιτεκτονική συνόλου εντολών.

Abstract

The thesis aims to study the techniques used in software analysis and debugging, by means of recording and tracing CPU instructions. As part of the thesis, a DTrace extension will be designed and developed, which will allow tracing of arbitrary assembly instructions within a given FreeBSD kernel function.

Keywords: FreeBSD, DTrace, Tracing, Debugging, Operating systems, C, Assembly, Instruction set architecture.

Περιεχόμενα

1	Εισαγωγή στα εργαλεία παρακολούθησης (tracing)	7
1.1	Παρακολούθηση στο FreeBSD	7
2	Εισαγωγή στο DTrace	8
2.1	Γλώσσα D	9
2.1.1	Probes	9
2.1.2	Providers	10
2.1.3	Μεταβλητές	11
2.1.4	Συναρτήσεις	11
2.1.5	Συσσωματώσεις (Aggregations)	11
2.1.6	Φίλτρα (Predicates)	13
2.1.7	CTF	13
2.1.8	DIF	15
2.2	fbt	15
2.3	Περισσότερα παραδείγματα χρήσης DTrace	17
3	kinst	19
3.1	Χρήση	19
3.2	Γιατί είναι χρήσιμο κάτι τέτοιο;	21
3.3	Παρακολούθηση εντολών assembly	21
3.4	Τραμπολίνο (Trampoline)	21
3.5	Υλοποίηση εντολών (Emulation)	22
4	Υλοποίηση	22
4.1	kinst	23
4.1.1	kinst.*	23
4.1.2	trampoline.c	24
4.1.3	amd64/kinst_isa.*	24
4.1.4	aarch64/kinst_isa.* και riscv/kinst_isa.*	26
4.2	Παρακολούθηση inline συναρτήσεων	28
4.2.1	Τι είναι οι inline συναρτήσεις και γιατί είναι δύσκολη η παρακολούθησή τους;	29
4.2.2	Πώς ο kinst λύνει αυτό το πρόβλημα	29
4.2.3	Πρότυπο DWARF	30
4.2.4	Υπολογισμός ορίων κλήσεων inline αντιγράφων	32
4.2.5	Εύρεση καλούσας συνάρτησης	34
4.2.6	Υπολογισμός των offsets entry και return	36
4.3	kinst tests	41
5	Πειράματα	42
5.1	Περιβάλλον εκτέλεσης	42
5.1.1	AMD64	42
5.1.2	ARM64	43
5.1.3	RISC-V	44
5.2	Μεταγλώττιση DTrace	45
5.3	Δοκιμές	45
5.3.1	AMD64	45
5.3.2	ARM64	47
5.3.3	RISC-V	49

6	Συμπεράσματα και προτάσεις μελλοντικών εργασιών	53
7	Βιβλιογραφία	54
	Συντομογραφίες	56

1 Εισαγωγή στα εργαλεία παρακολούθησης (tracing)

Η ανάπτυξη πυρήνων λειτουργικών συστημάτων είναι από την φύση της περίπλοκη. Ο πυρήνας δεν είναι ένα τυπικό γραφικό πρόγραμμα, αλλά ένα πρόγραμμα που τρέχει στο παρασκήνιο και είναι υπεύθυνο για την διαχείριση των πόρων του υπολογιστή, των συστημάτων αρχείων, του χρονοπρογραμματισμού των διεργασιών, την επικοινωνία του λειτουργικού συστήματος με το υλικό, και πολλές άλλες λειτουργίες χαμηλού επιπέδου. Ο προγραμματισμός λοιπόν ενός τέτοιου συστήματος περιλαμβάνει την κατανόηση υποσυστημάτων χαμηλού επιπέδου, τις πολύπλοκες διασυνδέσεις μεταξύ τους, η οποία είναι απαραίτητη για την αποσφαλμάτωση, όταν το σφάλμα προκύψει. Η αποσφαλμάτωση πυρήνων είναι επίσης μία ιδιάζουσα περίπτωση, διότι ακόμα και ένα απλό σφάλμα μπορεί να έχει ως αποτέλεσμα από απρόβλεπτη συμπεριφορά/δυσλειτουργία του λειτουργικού συστήματος, αλλοίωση αρχείων και υλικού, μέχρι και ολική κατάρρευση του λειτουργικού συστήματος. Στην περίπτωση τέτοιων σφαλμάτων, με άλλα λόγια, η αποσφαλμάτωση απαιτεί σύνθετη κατανόηση του πυρήνα, αλλά σε κάποιες περιπτώσεις μπορεί να είναι ακόμα και αδύνατη, ή τουλάχιστον πολύ περιορισμένη. Επιπλέον, προκειμένου να αποκτηθεί εξαρχής αυτή η κατανόηση του πυρήνα, η απλή ανάγνωση του κώδικά του, αν και προφανώς απαραίτητη, δεν είναι πάντα η βέλτιστη λύση, ειδικά όταν πρόκειται για πολύ περίπλοκα συστήματα που μπορεί να αποτελούνται από δεκάδες ή εκατοντάδες χιλιάδες γραμμές κώδικα, οι οποίες αλληλεπιδρούν με πολλαπλά συστήματα και δημιουργούν μία μεγάλη αλυσίδα πολυπλοκότητας, της οποίας η κατανόηση απαιτεί πιο εξεζητημένες τεχνικές.

Όλα αυτά καθιστούν αναγκαία την ύπαρξη εργαλείων που μπορούν να βοηθήσουν στην αποσφαλμάτωση, όμως εξίσου αναγκαία είναι και η ύπαρξη εργαλείων που βοηθούν στην κατανόηση της ροής εκτέλεσης του πυρήνα, δηλαδή της παρακολούθησής του. Με τον όρο tracing εννοούμε την παρακολούθηση της ροής εκτέλεσης ενός προγράμματος σε πραγματικό χρόνο (real-time), με σκοπό την εξαγωγή δεδομένων, που είναι χρήσιμα κυρίως στην αποσφαλμάτωση (debugging) του προγράμματος, στην καλύτερη κατανόηση της ροής εκτέλεσης, καθώς και στην εύρεση τυχόν σημείων που προκαλούν καθυστερήσεις (bottlenecks). Κατά μία έννοια το tracing μπορεί να θεωρηθεί ως μία πιο εξειδικευμένη και πιο δυναμική μορφή καταγραφής πληροφοριών (logging), διότι δίνεται η δυνατότητα στον χρήστη να εξάγει μη-συγκεκριμένες πληροφορίες από ένα πρόγραμμα, χωρίς να χρειάζεται αναγκαστικά η συγγραφή περαιτέρω πηγαίου κώδικα μέσα στο πρόγραμμα και η επαναμεταγλώττισή του, κάτι το οποίο, στην περίπτωση μεγάλων προγραμμάτων όπως ο πυρήνας ενός λειτουργικού συστήματος, τείνει να είναι μία χρονοβόρα και δυσκίνητη διαδικασία.

Προγράμματα παρακολούθησης χρησιμοποιούνται κατά κόρον από προγραμματιστές πυρήνων λειτουργικών συστημάτων και διαχειριστές συστημάτων (ασφάλεια, servers, βάσεις δεδομένων), και είναι διαθέσιμα σε όλα τα λειτουργικά συστήματα, ανοιχτού και μη κώδικα.

Συνοπτικά μεταξύ άλλων, μερικές από τις συνηθισμένες χρήσεις προγραμμάτων παρακολούθησης, είναι οι καταγραφή χρόνων εκτέλεσης συναρτήσεων, εξαγωγή ορισμάτων συναρτήσεων και τιμών καταχωρητών επεξεργαστή, η εμφάνιση στοίβας κλήσεων (call stack), παραγωγή γραφημάτων απεικόνισης εκτέλεσης, καταγραφή συμβάντων (event logging), εισαγωγή breakpoints, κατανόηση ροής εκτέλεσης, και πολλές άλλες.

Στην παρούσα εργασία θα ασχοληθούμε με το λειτουργικό σύστημα FreeBSD, το οποίο είναι το πιο δημοφιλές από την οικογένεια των BSD (Berkley Software Distribution) λειτουργικών συστημάτων. Εμφανίστηκε το 1993 ως κλώνος του UNIX και ο κώδικας του είναι ανοιχτός (open source). Έχει μικρότερο αριθμό χρηστών από τα λειτουργικά συστήματα Linux, ωστόσο, πάρα της μικρότερης κοινότητας του, διακρίνεται για την σταθερότητά του και την αποδοτικότητα του, αλλά και του προσεγμένου και καθαρού του πηγαίου κώδικα, και χρησιμοποιείται εκτενώς από διάφορες μεγάλες (και μη) εταιρίες για το στήσιμο των server τους.

1.1 Παρακολούθηση στο FreeBSD

Το FreeBSD παρέχει πληθώρα προγραμμάτων παρακολούθησης και καταγραφής [1], για διάφορους τομείς του λειτουργικού συστήματος. Ενδεικτικά, μερικά από αυτά τα εργαλεία είναι τα εξής:

Πρόγραμμα	Λειτουργία
top(1)	Πληροφορίες για την κατάσταση τρέχουσων διεργασιών.
netstat(1)	Κατάσταση δικτύου.
procstat(1)	Αναλυτικές πληροφορίες διεργασιών.
vmstat(8)	Πληροφορίες εικονικής μνήμης (virtual memory).
systat(1)	Στατιστικά συστήματος.
fstat(1)	Πληροφορίες ενεργών αρχείων.
sockstat(1)	Πληροφορίες ανοιχτών sockets.
lockstat(1)	Πληροφορίες locking εντός του πυρήνα.
plockstat(1)	Πληροφορίες pthread locking.
truss(1)	Παρακολούθηση κλήσεων συστήματος (system calls).
dtrace(1)	Πρόγραμμα δυναμικής παρακολούθησης.
ktrace(1)	Πρόγραμμα καταγραφής λειτουργιών πυρήνα κατά την εκτέλεση μίας διεργασίας.
pmcstat(8)	Μετρήσεις επίδοσης συστήματος.
ktrdump(8)	Εκτύπωση ktr(4) buffer.

Όλα τα παραπάνω εργαλεία χρησιμοποιούνται εκτενώς από τους χρήστες του FreeBSD, ωστόσο δύο βασικοί περιορισμοί μερικών από αυτά είναι ότι 1) δεν λειτουργούν σε πραγματικό χρόνο (real-time), 2) δεν είναι πάντα ευέλικτα, διότι κάνουν συγκεκριμένες «ερωτήσεις» για την κατάσταση του συστήματος που παρακολουθούν, αλλά και παράγουν συγκεκριμένες εξόδους με συγκεκριμένα δεδομένα. Κάτι τέτοιο είναι ιδανικό και αρκετό σε πολλές περιπτώσεις, όμως υπάρχουν και περιπτώσεις που ο διαχειριστής/προγραμματιστής χρειάζεται να κάνει «ερωτήσεις» οι οποίες δεν καλύπτονται από κάποιο υπάρχον πρόγραμμα, καθώς και να κάνει καταγραφή δεδομένων με πιο εξειδικευμένο τρόπο. Αυτά είναι μερικά από τα βασικά προβλήματα που λύνουν τα προγράμματα δυναμικής παρακολούθησης (dynamic tracing). Στο FreeBSD, το DTrace είναι το πιο συνηθισμένο εργαλείο δυναμικής παρακολούθησης σε πραγματικό χρόνο.

2 Εισαγωγή στο DTrace

Το DTrace [3] [4] [8] [6] [5] [7] είναι ένα framework δυναμικής παρακολούθησης (dynamic tracing framework) [2], το οποίο προήλθε από το λειτουργικό σύστημα Solaris το 2005 και έπειτα μεταφέρθηκε στο FreeBSD μεταξύ 2006 και 2008. Μεταξύ άλλων, χρησιμοποιείται για μετρήσεις επιδόσεων, αποσφαλμάτωση και παρακολούθηση σε πραγματικό χρόνο, καθώς και παρατήρηση συμπεριφοράς συγκεκριμένων υποσυστημάτων του λειτουργικού συστήματος. Το DTrace είναι ιδιαίτερα ισχυρό χάρη στην γλώσσα D (βλέπε ενότητα 2.1), η οποία περιέχει μεγάλη γκάμα δυνατοτήτων και χρησιμοποιείται για την σύνταξη D scripts, τα οποία φορτώνονται δυναμικά στον πυρήνα του λειτουργικού συστήματος, εκτελούν όποια λειτουργία τούς δόθηκε, και έπειτα αποφορτώνονται.

Πάνω στο θέμα των D scripts, είναι σημαντικό να σημειωθεί ότι τα scripts αυτά, αν και φορτώνονται και εκτελούνται στον πυρήνα, δεν μπορούν να αλλάξουν την λογική/ροή της εκτέλεσης του ή τον ίδιο τον κώδικά του πυρήνα¹. Στην ουσία, αυτό που κάνει ένα D script αφού φορτωθεί στον πυρήνα, είναι να συλλέξει πληροφορίες από τον πυρήνα, να τις επεξεργαστεί, και να τις μεταφέρει πίσω στον χρήστη. Αυτό σημαίνει ότι τα D scripts είναι ασφαλή για χρήση σε συστήματα που χρησιμοποιούνται σε παραγωγή, το οποίο είναι και αυτό που κάνει το DTrace ιδιαίτερα εύχρηστο.

Η χρήση του DTrace γίνεται είτε μέσω του προγράμματος `dtrace(1)`, είτε έμμεσα μέσω επιπλέον διαφόρων προγραμμάτων, όπως το `lockstat(1)`, `truss(1)`, και άλλα.

¹Μόνη εξαίρεση σε αυτό είναι ορισμένες «καταστροφικές», όπως ονομάζονται, συναρτήσεις (βλέπε ενότητα 2.1.4), οι οποίες όμως χρησιμοποιούνται μόνο σκόπιμα.

2.1 Γλώσσα D

Η D είναι η γλώσσα προγραμματισμού που χρησιμοποιείται για την σύνταξη DTrace scripts, η οποία παρομοιάζει την C και την AWK. Η δομή ενός D script είναι αντίστοιχη με αυτή της AWK:

```
<options>

<variables, structs, ...>

<provider>:<module>:<function>:<name>
/<predicate>/
{
    <actions>
}

[...]
```

Αξίζει να σημειωθεί ότι δεν είναι απαραίτητο να ορίζονται όλα τα πεδία — το παραπάνω απλώς δείχνει την γενική δομή ενός D script.

2.1.1 Probes

Στην ορολογία του DTrace, probe ονομάζεται κάθε σημείο ή κομμάτι κώδικα προς παρακολούθηση. Αυτά τα σημεία μπορούν να αντιστοιχούν σε συμβάντα (events), κατηγορίες συμβάντων σε κάποιο υποσύστημα του kernel ή του userspace, και συγκεκριμένα σημεία εντός συναρτήσεων. Τα probes επιλέγονται και ενεργοποιούνται από τον χρήστη κατά την εκτέλεση του προγράμματος `dtrace(1)`, το οποίο στην συνέχεια φορτώνει το D script στον πυρήνα, και στο οποίο επιστρέφονται — από το thread που εκτέλεσε τον κώδικα προς παρακολούθηση, με άλλα λόγια όταν πυροδοτηθεί το probe — οι πληροφορίες που συλλέχθηκαν για κάθε probe, προκειμένου να τυπωθούν και να αξιοποιηθούν από τον χρήστη.

Τα probes ορίζονται ως πλειάδες όπου κάθε πεδίο χωρίζεται με `..`. Τα πεδία είναι τα εξής:

```
<provider>:<module>:<function>:<name>
```

Πιο συγκεκριμένα:

- **<provider>**: Όνομα του συνόλου/namespace στο οποίο ανήκει το probe που θέλουμε να δημιουργήσουμε. Περισσότερα στην ενότητα 2.1.2.
- **<module>** (προαιρετικό): Υποσύστημα/module. Σε ορισμένους providers αυτό αντιστοιχεί σε κανονικά kernel modules (`kernel`, `sound`, κλπ.), σε άλλους σε κατηγορίες ABI (`freebsd`, `freebsd32`), και σε άλλους σε νοητές υποκατηγορίες.
- **<function>** (προαιρετικό): Συνήθως αναφέρεται σε συναρτήσεις C του προγράμματος/υποσυστήματος που παρακολουθείται, αλλά δεν ισχύει πάντα αυτό.
- **<name>** (προαιρετικό): Όνομα probe.

Τα πεδία `<module>`, `<function>` και `<name>` επίσης μπορούν να οριστούν κάνοντας χρήση wildcarding, δηλαδή με την χρήση του χαρακτήρα `*`, όπως συμβαίνει και στις κανονικές εκφράσεις (regex). Για παράδειγμα αν θέλουμε να παρακολουθήσουμε όλες τις συναρτήσεις που ξεκινάνε με το όνομα `open` (δηλαδή `open`, `openat`, κλπ.), μπορούμε να ορίσουμε το πεδίο ως `open*`.

Πολλοί providers δημιουργούν τα probes τους κατά την φόρτωση τους. Η εντολή `dtrace(1)` μπορεί να τυπώσει όλα τα διαθέσιμα probes (χωρίς να τα ενεργοποιήσει) με την επιλογή `-l`:

```
# dtrace -l
  ID    PROVIDER      MODULE                                FUNCTION NAME
  1      dtrace                                     BEGIN
  2      dtrace                                     END
  3      dtrace                                     ERROR
  4      fbt          kernel                                camstatusentrycomp entry
  5      fbt          kernel                                camstatusentrycomp return
  6      fbt          kernel                                cam_compat_handle_0x18 entry
  7      fbt          kernel                                cam_compat_handle_0x18 return
[...]
```

Επίσης μπορεί να γίνει φιλτράρισμα των probes βάσει:

Φίλτρο	Επιλογή dtrace(1)
Probe ID	-i <probeid>
Provider	-P <provider>
Module	-m <module>
Συνάρτησης	-f <function>
Ονόματος probe	-n <probename>

2.1.2 Providers

Ως providers ορίζονται τα διάφορα modules που είναι υπεύθυνα για συγκεκριμένες κατηγορίες παρακολούθησης — με άλλα λόγια, μπορούμε να ορίσουμε έναν provider ως ένα σύνολο ή namespace λογικά συνδεδεμένων probes. Πρακτικά, οι DTrace providers είναι είτε συνηθισμένα kernel modules όταν η παρακολούθηση αφορά τον πυρήνα, είτε userspace modules όταν αφορούν παρακολούθηση υποσυστημάτων του userspace.

Στο FreeBSD υπάρχουν αρκετοί providers που καλύπτουν διάφορα υποσυστήματα του kernel και του userspace αντίστοιχα, ένας εκ των οποίων είναι και ο **kinst** (βλέπε ενότητα 3), στον οποίο έχει βασιστεί το αντικείμενο της παρούσας εργασίας. Ενδεικτικά, μερικοί από τους σημαντικούς providers που βρίσκονται στο FreeBSD:

Provider	Είδος/τομέας παρακολούθησης
dtrace	Ορίζει τα probes BEGIN, END και ERROR
profile	Περιέχει probes που πυροδοτούνται βάσει ορισμένου χρονικού διαστήματος.
syscall	Κλήσεις συστήματος
dtmalloc	Δεσμεύσεις μνήμης στον πυρήνα
proc	Συμβάντα σχετικά με διεργασίες
ip	Συμβάντα σχετικά με τα πρωτόκολλα IPv4 και IPv6
tcp/udp	Συμβάντα TCP ή UDP
sched	Συμβάντα χρονοπρογραμματισμού (scheduling)
fbt	Αρχή ή/και τέλος συναρτήσεων πυρήνα
kinst	Οποιαδήποτε εντολή assembly εντός συναρτήσεων πυρήνα, καθώς και inline συναρτήσεις
lockstat	Locking εντός του πυρήνα
io	I/O δίσκων
sdt	Στατικά probes εντός του πηγαίου κώδικα του πυρήνα
usdt	Όπως ο sdt αλλά για το userspace
pid	Όπως ο fbt αλλά για το userspace

2.1.3 Μεταβλητές

Η γλώσσα D περιλαμβάνει built-in μεταβλητές οι οποίες μπορούν να χρησιμοποιηθούν στα scripts. Μερικές συνηθισμένες και χρήσιμες μεταβλητές είναι οι παρακάτω:

Μεταβλητή	Περιγραφή
args[]	Πίνακας ορισμάτων παρακολουθούμενου probe κωδικοποιημένα με CTF.
arg0, ..., arg9	Ορίσματα παρακολουθούμενου probe σε μοφή 64-bit unsigned integer. Μόνο τα πρώτα 10 ορίσματα είναι διαθέσιμα μέσω αυτών των μεταβλητών.
execargs	Ορίσματα τρέχουσας διεργασίας (curthread->td_proc->p_args).
execname	Όνομα τρέχουσας διεργασίας (curthread->td_proc->p_comm).
gid	Group ID τρέχουσας διεργασίας.
pid	Process ID τρέχουσας διεργασίας.
uid	User ID τρέχουσας διεργασίας.
tid	Thread ID, είτε του kernel thread, είτε της διεργασίας.
regs[]	Τιμές τρέχοντος CPU.
cpu	ID τρέχοντος CPU.
errno	Τιμή errno(2) της τελευταίας κλήσης συστήματος στο τρέχον thread.
curthread	Pointer στην global δομή curthread του τρέχοντος CPU.
timestamp	Nanoseconds από την εκκίνηση του υπολογιστή.

Πέρα από τις μεταβλητές που παρέχονται από την γλώσσα D ωστόσο, ο χρήστης μπορεί να ορίσει και δικές του — global και τοπικές — μεταβλητές.

2.1.4 Συναρτήσεις

Όπως και με τις μεταβλητές, η γλώσσα D επίσης περιλαμβάνει και built-in συναρτήσεις. Μερικές συνηθισμένες και χρήσιμες συναρτήσεις είναι οι παρακάτω:

Συνάρτηση	Περιγραφή
printf(...), printa(...), print(...)	Εκτύπωση κειμένου.
stack(), ustack()	Στοίβα κλήσεων kernel/userspace του thread που εκτελεί το τρέχον probe.
breakpoint()	Ορισμός kernel breakpoint και μεταφορά ελέγχου στον kernel debugger ddb(4).
panic()	Πρόκληση kernel panic.

2.1.5 Συσσωματώσεις (Aggregations)

Οι συσσωματώσεις είναι ένας τρόπος σύλλεξης και οργάνωσης δεδομένων κατά την παρακολούθηση, και εκτύπωσής τους στο τέλος της παρακολούθησης, με διάφορους τρόπους κατανομής τους ή ανάλυσής τους.

Η γενική σύνταξη των συσσωματώσεων είναι η παρακάτω:

```
@aggname[<key1>, ...] = aggfunc(args);
```


Με τον χαρακτήρα @ δηλώνουμε μία συσσωμάτωση, στην οποία μπορούμε επίσης να δώσουμε και κάποιο όνομα. Η σύλλεξη των δεδομένων πολύ συχνά γίνεται βάσει ενός ή παραπάνω κλειδιών. Τέλος για κάθε συσσωμάτωση ορίζεται και μία συνάρτηση, η οποία δρα πάνω στα συλλεγμένα δεδομένα.

Μερικές συνηθισμένες συναρτήσεις είναι οι εξής:

Συνάρτηση	Περιγραφή
quantize(val)	Κατανομή συχνότητας δεδομένων σε δυνάμεις του 2.
lquantize(val, factor, low, high, nsteps)	Γραμμική κατανομή συχνότητας δεδομένων.
min(val)	Ελάχιστο.
max(val)	Μέγιστο.
sum(val)	Άθροισμα.
count()	Μέτρηση.
avg(val)	Υπολογισμός μέσου όρου.
stddev(val)	Τυπική απόκλιση.

Οι συσσωματώσεις τυπώνονται είτε στο τέλος της παρακολούθησης, είτε με την αποστολή του POSIX σήματος SIGINFO, το οποίο μπορεί να γίνει εύκολα με τον συνδυασμό πληκτρολογίου CTRL-T στην γραμμή εντολών.

Για παράδειγμα, η παρακάτω εντολή δηλώνει μία συσσωμάτωση με όνομα progs, συλλέγει όλες τις κλήσεις στα write(2) system calls και τις οργανώνει/αποθηκεύει βάσει του ονόματος του προγράμματος που τις εκτελεί (μεταβλητή execname), και τυπώνει το όνομα του κάθε προγράμματος στα αριστερά, και τον αριθμό των κλήσεων που πραγματοποίησε στα δεξιά, με την χρήση της συνάρτησης count(), η οποία θα μετρήσει πόσες φορές εμφανίζεται κάθε κλειδί.

```
# dtrace -n 'syscall::write*:entry { @progs[execname] = count(); }'
dtrace: description 'syscall::write*:entry ' matched 3 probes
^T
load: 0.29  cmd: dtrace 25176 [uwait] 4.35r 0.04u 0.11s 0% 42372k
  irssi                                     1
  st                                        1
  librewolf                                5
  dwm                                       8
  calcurse                                 13
  slstatus                                 44
  Xorg                                     680
  mpv                                      798

^C

  irssi                                     1
  tmux                                     6
  st                                       9
  dtrace                                  11
  dwm                                    14
  librewolf                              18
  calcurse                               22
  slstatus                               77
  Xorg                                   1043
  mpv                                   1394
```

Ο λόγος που η συσσωμάτωση τυπώθηκε δύο φορές είναι διότι το πρώτο τύπωμα έγινε με την αποστολή του σήματος SIGINFO, ενώ η δεύτερη έγινε λόγω του ότι τελείωσε η παρακολούθηση.

2.1.6 Φίλτρα (Predicates)

Τα predicates είναι ένας γρήγορος τρόπος φιλτραρίσματος των παρακολουθούμενων probes, με παρόμοιο τρόπο και σύνταξη όπως στην γλώσσα AWK. Κάθε predicate αξιολογείται κατά την πυροδότηση του probe και αν η συνθήκη που έχει οριστεί είναι αληθής, τότε εκτελείται και το σώμα του probe, ειδικά το συγκεκριμένο probe αγνοείται. Εντός της συνθήκης που ορίζεται στο probe μπορεί να γίνει χρήση μεταβλητών.

Για παράδειγμα, αν θέλαμε να φιλτράρουμε τις αποστολές πακέτων IP, μόνο όταν προέρχονται από το firefox, θα ορίζαμε το παρακάτω probe και predicate:

```
ip:::send
/execname == "firefox"/
{
}
```

2.1.7 CTF

Υπάρχουν πολλές περιπτώσεις που ο χρήστης θέλει να εξάγει από το κομμάτι που παρακολουθεί πληροφορίες σχετικά με κάποια δομή δεδομένων του πυρήνα που παρακολουθεί, ορίσματα των probes, και ούτω καθεξής. Προκειμένου όμως να μπορέσει ένα probe να συλλέξει αυτές τις πληροφορίες από τον πυρήνα και να είναι χρήσιμες και κατανοητές για τον χρήστη, πρέπει επίσης να γνωρίζει τον τύπο και την αναπαράστασή τους. Αντίστοιχο πρόβλημα αντιμετωπίζουν και τα πρόγραμμα αποσφαλμάτωσης (debuggers). Στην περίπτωση πολλών debuggers (όπως του GDB) χρησιμοποιείται ένα πρότυπο κωδικοποίησης πληροφοριών που ονομάζεται «DWARF», καθώς επίσης χρησιμοποιείται και για την υλοποίηση της παρούσας εργασίας και στο οποίο γίνεται αναφορά στην ενότητα 4.2.3. Ωστόσο αυτό το πρότυπο είναι αρκετά περίπλοκο και «μεγάλο» για την περίπτωση του DTrace, το οποίο χρειάζεται μόνο να αναπαραστήσει τύπους δεδομένων (integers, structs, unions, κλπ.) της γλώσσας C.

Το πρότυπο που χρησιμοποιείται από το DTrace είναι το CTF [29], το οποίο μπορεί είναι ένα υποσύνολο του DWARF, και στο FreeBSD χρησιμοποιείται κατά κύριο λόγο από το DTrace.

Το τεχνικό κομμάτι του CTF δεν αφορά το κομμάτι της εργασίας, οπότε δεν χρειάζεται να γίνει αναλυτική αναφορά σε αυτό. Η εμφάνιση του γραφήματος τύπων CTF μπορεί να γίνει με την εντολή `ctfdump(1)`. Για παράδειγμα, η παρακάτω εντολή θα τυπώσει το γράφημα τύπων CTF για τον πυρήνα του FreeBSD:

```
# ctfdump -t /boot/kernel/kernel

- Types -----

<1> INTEGER char encoding=SIGNED CHAR offset=0 bits=8
<2> INTEGER long encoding=SIGNED offset=0 bits=64
[3]
<4> INTEGER unsigned int encoding=0x0 offset=0 bits=32
<5> TYPEDEF uInt refers to 4
[6] INTEGER unsigned char encoding=CHAR offset=0 bits=8

[...]
```

Το DTrace κάνει χρήση αυτής της κωδικοποίησης τύπων δεδομένων C, ώστε να μπορεί ο χρήστης να έχει πρόσβαση στους τύπους δεδομένων του πυρήνα εντός του D script. Για παράδειγμα, ας υποθέσουμε την συνάρτηση `vm_fault()` (`sys/vm/vm_fault.c`):

```

int
vm_fault(vm_map_t map, vm_offset_t vaddr, vm_prot_t fault_type,
         int fault_flags, vm_page_t *m_hold)
{
    [...]
}

```

Η παρακάτω εντολή τυπώνει το πρώτο όρισμα της συνάρτησης `vm_fault()`, δηλαδή το `vm_map_t map`, κάθε φορά που ο πυρήνας εισέρχεται στην συνάρτηση:

```

# dtrace -n 'fbt::vm_fault:entry { print(*args[0]); }'
dtrace: description 'fbt::vm_fault:entry ' matched 1 probe
CPU      ID                      FUNCTION:NAME
 0  50203                      vm_fault:entry struct vm_map {
    struct vm_map_entry header = {
        struct vm_map_entry *left = 0xffffffff809a5098ea0
        struct vm_map_entry *right = 0xffffffff8002ecad840
        vm_offset_t start = 0x8000000000000000
        vm_offset_t end = 0x1000
        vm_offset_t next_read = 0
        vm_size_t max_free = 0
        union vm_map_object object = {
            struct vm_object *vm_object = 0
            struct vm_map *sub_map = 0
        }
        vm_ooffset_t offset = 0
        vm_eflags_t eflags = 0x80000
        vm_prot_t protection = 0
        vm_prot_t max_protection = 0
        vm_inherit_t inheritance = '\0'
        uint8_t read_ahead = 0
        int wired_count = 0
        struct ucred *cred = 0
        struct thread *wiring_thread = 0
    }
    struct sx lock = {
        struct lock_object lock_object = {
            const char *lo_name = 0xffffffff811cbe73
            u_int lo_flags = 0x2330000
            u_int lo_data = 0
            struct witness *lo_witness = 0
        }
        volatile uintptr_t sx_lock = 0x1
    }
    struct mtx system_mtx = {
        struct lock_object lock_object = {
            const char *lo_name = 0xffffffff810d7c8f
            u_int lo_flags = 0x1430000
            u_int lo_data = 0
            struct witness *lo_witness = 0
        }
    }
}

```

```

    }
    volatile uintptr_t mtx_lock = 0
}
int nentries = 0x99
vm_size_t size = 0x6ab8000
u_int timestamp = 0x385
u_char needs_wakeup = 0
u_char system_map = 0
vm_flags_t flags = 0x98
vm_map_entry_t root = 0xfffff805835c5d20
pmap_t pmap = 0xfffff8000e536ad0
vm_offset_t anon_loc = 0x3869fde00000
int busy = 0
}
^C

```

Όπως βλέπουμε, χάρη στην κωδικοποίηση CTF, μπορούμε να τυπώσουμε το όρισμα που είναι τύπου `vm_map_t`, ενός τύπου ορισμένου στον πυρήνα του FreeBSD, και να δούμε τα περιεχόμενα του σε αναγνώσιμη μορφή κατά την στιγμή της πυροδότησης του probe.

2.1.8 DIF

Για να εισαχθεί ένα D script στον πυρήνα του λειτουργικού συστήματος, χρησιμοποιείται η κωδικοποίηση DIF. Η κωδικοποίηση αυτή έχει ως στόχο την μεταγλώττιση του D script σε ενδιάμεσο byte κώδικα (byte code), ο οποίος μπορεί να εκτελεστεί από το kernel. Αφού το script έχει μεταγλωττιστεί, φορτώνεται στο kernel από το DTrace, και εκτελείται για τα probes που έχουν οριστεί όταν αυτά ενεργοποιηθούν.

Η κωδικοποίηση DIF για ένα δεδομένο D script μπορεί να εμφανιστεί χρησιμοποιώντας την επιλογή `-S` στην εντολή `dtrace(1)`:

```

# dtrace -Sn 'ip:::send { printf("%s", execname); exit(0); }'

DIFO 0x2241cb7f41c0 returns string (unknown) by ref (size 256)
OFF OPCODE      INSTRUCTION
00: 29011801    ldgs DT_VAR(280), %r1          ! DT_VAR(280) = "execname"
01: 23000001    ret  %r1

NAME            ID    KND SCP FLAG TYPE
execname        280   scl glb r    string (unknown) by ref (size 256)

DIFO 0x2241cb7f4230 returns D type (integer) (size 4)
OFF OPCODE      INSTRUCTION
00: 25000001    setx DT_INTEGER[0], %r1          ! 0x0
01: 23000001    ret  %r1
dtrace: description 'ip:::send ' matched 1 probe
^C

```

Αντίστοιχα στο Linux, χρησιμοποιείται η κωδικοποίηση eBPF.

2.2 fbt

Ο `fbt` παρακολουθεί την έναρξη ή/και το τέλος μίας kernel συνάρτησης με την χρήση δύο probes, `entry` και `return` αντίστοιχα.

Για κάθε probe ο fbt εκθέτει ορίσματα. Για τα entry probes τα ορίσματα αντιστοιχούν στα ορίσματα της συνάρτησης που παρακολουθείται, όπως ορίζονται στον κώδικά της. Για παράδειγμα:

```
# dtrace -lvn 'fbt:kernel:malloc:entry'
  ID    PROVIDER      MODULE                FUNCTION NAME
30144      fbt        kernel                malloc entry
[...]
  Argument Types
    args[0]: size_t
    args[1]: struct malloc_type *
    args[2]: int
```

Για τα return probes τα ορίσματα δεν αντιστοιχούν στα ορίσματα της συνάρτησης, αλλά είναι μόνο δύο, και αντιστοιχούν σε:

- args[0]: Offset στην εντολή επιστροφής.
- args[1]: Τιμή επιστροφής (return value) της συνάρτησης, όπως ορίζεται στον κώδικά της.

```
# dtrace -lvn 'fbt:kernel:malloc:return'
  ID    PROVIDER      MODULE                FUNCTION NAME
30145      fbt        kernel                malloc return
[...]
  Argument Types
    args[0]: int
    args[1]: void *
```

Για παράδειγμα:

```
# dtrace -n 'fbt:kernel:malloc:entry { printf("alloc size: %d", args[0]); }'
dtrace: description 'fbt:kernel:malloc:entry ' matched 1 probe
CPU    ID                FUNCTION:NAME
  5  30144                malloc:entry alloc size: 608
  5  30144                malloc:entry alloc size: 608
  5  30144                malloc:entry alloc size: 88
  5  30144                malloc:entry alloc size: 388
  5  30144                malloc:entry alloc size: 64
  5  30144                malloc:entry alloc size: 96
[...]

# dtrace -n 'fbt:kernel:malloc:return { printf("alloc addr: %p", args[1]); }'
dtrace: description 'fbt:kernel:malloc:return ' matched 1 probe
CPU    ID                FUNCTION:NAME
  12  30145                malloc:return alloc addr: fffff80647b7e400
  12  30145                malloc:return alloc addr: fffff80647b7e400
   6  30145                malloc:return alloc addr: fffff8089a666660
   5  30145                malloc:return alloc addr: fffff80003fbe4f0
   2  30145                malloc:return alloc addr: fffff807e9977df0
[...]
```

Ο fbt δημιουργεί όλα τα probes κατά την φόρτωση του module, και μπορούν να τυπωθούν με την χρήση της παρακάτω εντολής dtrace -l -P fbt.

2.3 Περισσότερα παραδείγματα χρήσης DTrace

Παρακολούθηση ανοιγμάτων αρχείων (`open(2)` syscall) και εκτύπωση του προγράμματος που εκτελεί το άνοιγμα:

```
# dtrace -n 'syscall::open*:entry { printf("%s", execname); }'
dtrace: description 'syscall::open*:entry ' matched 4 probes
CPU      ID                      FUNCTION:NAME
  1  69846                      open:entry sshd
  5  69846                      open:entry sshd-session
  5  69846                      open:entry sshd-session
  5  69846                      open:entry sshd-session
```

Μέτρηση χρόνου εκτέλεσης (σε nanoseconds) της kernel συνάρτησης `vm_fault`. Η συνάρτηση `quantize()` θα εκτυπώσει στο τέλος της παρακολούθησης ένα ιστόγραμμα όπου στην αριστερή στήλη εμφανίζονται οι χρόνοι εκτέλεσης (στρογγυλοποιημένοι σε δυνάμεις του δύο), και στην δεξιά στήλη το πλήθος των κλήσεων για κάθε χρόνο εκτέλεσης:

```
# cat > timing.d
fbt::vm_fault:entry
{
    self->ts = timestamp;
}

fbt::vm_fault:return
/self->ts/
{
    @ = quantize(timestamp - self->ts);
    self->ts = 0;
}
^D

# dtrace -s timing.d
dtrace: script 'timing.d' matched 2 probes
^C

      value  ----- Distribution -----  count
      256 |                                     0
      512 |@@@@@                               272
     1024 |@                                   42
     2048 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ 1671
     4096 |@                                   44
     8192 |                                     21
    16384 |                                     9
    32768 |                                     1
    65536 |
```

Παρακολούθηση της kernel συνάρτησης `malloc(9)` και εκτύπωση του προγράμματος που κάλεσε την `malloc()`, την αρχική διεύθυνση του κομματιού μνήμης που δεσμεύθηκε, το μέγεθος της δέσμευσης σε bytes, καθώς και την στοίβα κλήσεων (call stack):

```
# cat > malloc.d
```

```

fbt::malloc:entry
{
    self->bytes = args[0];
}

fbt::malloc:return
/self->bytes/
{
    printf("%s addr=%p bytes=%d", execname, args[1], self->bytes);
    self->bytes = 0;
    stack();
}
^D

# dtrace -s malloc.d
dtrace: script 'malloc.d' matched 2 probes
CPU      ID                      FUNCTION:NAME
  0   30145                      malloc:return dtrace addr=ffffff805c90b7800 bytes=608
                                dtrace.ko'dtrace_dof_create+0x18
                                dtrace.ko'dtrace_ioctl+0x970
                                kernel'devfs_ioctl+0xcb
                                kernel'vn_ioctl+0xce
                                kernel'devfs_ioctl_f+0x1e
                                kernel'kern_ioctl+0x255
                                kernel'sys_ioctl+0xff
                                kernel'amd64_syscall+0x10e
                                kernel'0xffffffff80fb312b

  3   30145                      malloc:return acpi_thermal addr=ffffff809ddc3f580 bytes=88
                                kernel'AcpiOsAllocateZeroed+0xe
                                kernel'AcpiEvaluateObject+0x27
                                kernel'acpi_GetInteger+0x3e
                                kernel'acpi_tz_get_temperature+0x21
                                kernel'acpi_tz_monitor+0x19
                                kernel'acpi_tz_thread+0x18c
                                kernel'fork_exit+0x7f
                                kernel'0xffffffff80fb387e

 13   30145                      malloc:return devd addr=ffffff80a42780d00 bytes=256
                                kernel'sysctl_handle_string+0x4e
                                kernel'sysctl_hostname+0xb0
                                kernel'sysctl_root_handler_locked+0x91
                                kernel'sysctl_root+0x23e
                                kernel'userland_sysctl+0x188
                                kernel'sys___sysctl+0x60
                                kernel'amd64_syscall+0x10e
                                kernel'0xffffffff80fb312b
^C

```

Ακόμη περισσότερα χρήσιμα και πιο εξεζητημένα παραδείγματα μπορεί κανείς να βρει παρακάτω:

- Brendan Gregg: The DTrace One-Liner Tutorial

- Brendan Gregg: DTrace One-Liners
- /usr/share/examples/dtrace

3 kinst

Ο **kinst** είναι ένας νέος DTrace provider, ο οποίος αναπτύχθηκε από τον συντάκτη της παρούσας εργασίας, σε συνεργασία με τον Mark Johnston, ως μέρος του FreeBSD. Δημιουργήθηκε με στόχο να αντιμετωπίσει τους περιορισμούς του **fbt** (βλέπε ενότητα 2.2), δηλαδή την έλλειψη δυνατότητας παρακολούθησης inline συναρτήσεων, καθώς και γενικότερα της πιο εξειδικευμένης παρακολούθησης πέραν της αρχής και του τέλους μίας συνάρτησης. Υποστηρίζεται στις αρχιτεκτονικές AMD64, ARM64 και RISC-V.

Το όνομα "kinst" είναι εμπνευσμένο από την δημοσίευση των Tamches & Miller [2], στην οποία κάνουν αναφορά στο πειραματικό εργαλείο παρακολούθησης που ανέπτυξαν, ως "KernInst".

Ο **kinst** δημιουργεί τα probes κατόπιν αιτήματος (on-demand) και όχι κατά το φόρτωμα του module, όπως συμβαίνει στην περίπτωση του **fbt**, ο οποίος δημιουργεί όλα τα probes κατά την φόρτωσή του. Ο λόγος που συμβαίνει αυτό είναι ότι σε αντίθεση με τον **fbt**, ο οποίος για κάθε συνάρτηση που μπορεί να παρακολουθήσει, δημιουργεί δύο probes (**entry** και **return**), ο **kinst** μπορεί να δημιουργήσει ακόμα και χιλιάδες probes για κάθε συνάρτηση, ανάλογα με το πόσες εντολές assembly περιέχει, κάτι το οποίο σε καμία περίπτωση δεν είναι αποδοτικό από άποψη διαχείρισης χωρητικότητας. Ένα αποτέλεσμα αυτής της προσέγγισης ωστόσο είναι ότι η εντολή `dtrace -l -P kinst` δεν τυπώνει κανένα probe.

3.1 Χρήση

Ο **kinst** δέχεται τις παρακάτω τρεις συντάξεις:

- `kinst::<function>`:

Με αυτή την σύνταξη παρακολουθούνται όλες οι εντολές assembly της συνάρτησης `<function>`. Για παράδειγμα:

```
# dtrace -n 'kinst::vm_fault:'
dtrace: description 'kinst::vm_fault:' matched 1481 probes
CPU      ID                FUNCTION:NAME
  1  71225                vm_fault:0
  1  71226                vm_fault:1
  1  71227                vm_fault:4
  1  71228                vm_fault:6
  1  71229                vm_fault:8
  1  71230                vm_fault:10
  1  71231                vm_fault:12
  1  71232                vm_fault:13
  1  71233                vm_fault:20
[...]
```

Όπως φαίνεται στην έξοδο της παραπάνω εντολής, ο **kinst** έφτιαξε probes για 1481 εντολές εντός της συνάρτησης `vm_fault()`. Παρατηρώντας το disassembly της `vm_fault()`, για παράδειγμα με την χρήση του GDB [27], βλέπουμε ότι το 1481 αντιστοιχεί στον συνολικό αριθμό εντολών της συνάρτησης:

```
# kgdb
(kgdb) set logging on
(kgdb) set pagination off
```



```
(kgdb) disas vm_fault
[...]
(kgdb) exit
# grep 0x gdb.txt | wc -l
1481
```

Στα συγκεκριμένα παραδείγματα χρησιμοποιείται ο KGDB [28], μία παραλλαγή του GDB που παρέχεται από το FreeBSD, με μερικές επιπλέον λειτουργίες για την καλύτερη αποσφαλμάτωση του πυρήνα.

- `kinst::<function>:<instruction>`

Με αυτή την σύνταξη παρακολουθείται εντός της συνάρτησης `<function>` μόνο η εντολή που βρίσκεται στο `offset` που ορίστηκε στο πεδίο `<instruction>`. Το `offset` αυτό μπορεί να βρεθεί στο `disassembly` της συνάρτησης. Για παράδειγμα:

```
# kgdb
(kgdb) disas /r vm_fault
Dump of assembler code for function vm_fault:
0xffffffff80f4e470 <+0>:    55      push    %rbp
0xffffffff80f4e471 <+1>:    48 89 e5    mov     %rsp,%rbp
0xffffffff80f4e474 <+4>:    41 57      push    %r15
[...]
```

Αν υποτεθεί ότι θέλουμε να παρακολουθήσουμε την τρίτη εντολή (`push %r15`) εντός της συνάρτησης `vm_fault()`, στην δεύτερη στήλη βλέπουμε ότι η εντολή αυτή βρίσκεται στο `offset 4`. Δίνοντας το `offset` αυτό στον `kinst`, μπορούμε να παρακολουθήσουμε μόνο αυτή την εντολή όποτε εκτελείται. Επιπλέον θα τυπώσουμε και τα περιεχόμενα του καταχωρητή `RSI`:

```
# dtrace -n 'kinst::vm_fault:4 {printf("%#x", regs[R_RSI]);}'
dtrace: description 'kinst::vm_fault:4 ' matched 1 probe
CPU      ID                FUNCTION:NAME
  1  71225                vm_fault:4 0x10652b411000
  1  71225                vm_fault:4 0x10652c341000
  1  71225                vm_fault:4 0x10652c342000
  1  71225                vm_fault:4 0x10652c343000
[...]
```

- `kinst::<inline_function>:<entry|return>`

Με αυτή την σύνταξη παρακολουθείται η αρχή (`entry`) ή το τέλος (`return`) μίας *inline* συνάρτησης `<inline_function>`. Για παράδειγμα:

```
# dtrace -n 'kinst::critical_enter:return'
dtrace: description 'kinst::critical_enter:return' matched 130 probes
CPU      ID                FUNCTION:NAME
  1  71024                spinlock_enter:53
  0  71024                spinlock_enter:53
  1  70992                uma_zalloc_arg:49
  1  70925  malloc_type_zone_allocated:21
  1  70994                uma_zfree_arg:365
  1  70924                malloc_type_freed:21
  1  71024                spinlock_enter:53
```

```
0 71024          spinlock_enter:53
0 70947          _epoch_enter_preempt:122
0 70949          _epoch_exit_preempt:28
[...]
```

3.2 Γιατί είναι χρήσιμο κάτι τέτοιο;

Σε αντίθεση με τον `fbt`, η δυνατότητα παρακολούθησης οποιασδήποτε μεμονωμένης εντολής `assembly` δίνει στον προγραμματιστή την επιλογή παρακολούθησης κομματιών κώδικα ή συγκεκριμένων σημείων εντός μίας συνάρτησης — `if-statements`, `loops`, εντόπιση διαδρομών κώδικα κλπ. Αυτό είναι ιδιαίτερα χρήσιμο για μεγάλες συναρτήσεις με πολλές διάφορες διαδρομές, όπου η ροή του κώδικα δεν είναι πάντα ξεκάθαρη ή εύκολη να κατανοηθεί.

Για παράδειγμα, αν ο προγραμματιστής έχει εντοπίσει κάποιο κομμάτι κώδικα για το οποίο δεν είναι σίγουρος αν εκτελείται ποτέ ή όχι, τότε μπορεί να χρησιμοποιήσει τον `kinst` για την παρακολούθηση εκείνου ακριβώς του σημείου. Αντίστοιχα, μπορεί να υπάρχει κώδικας που εκτελείται μόνο υπό προϋποθέσεις και ο προγραμματιστής θέλει να σιγουρευτεί αν και πότε εκτελείται.

Άλλο ένα βασικό σημείο του `kinst` είναι ότι θέτει την βάση για την υλοποίηση της παρακολούθησης `inline` συναρτήσεων, όπως αναλύεται στην ενότητα 4.2.

3.3 Παρακολούθηση εντολών `assembly`

Οι πληροφορίες των probes στέλνονται από το `dtrace(1)` στον `kinst` μέσω της `libdtrace`, χρησιμοποιώντας την ψευδοσυσκευή `/dev/dtrace/kinst`. Ο `kinst` είναι υπεύθυνος για τον εντοπισμό της συνάρτησης στο `kernel`, το `disassembly` της και την δημιουργία μίας εσωτερικής δομής δεδομένων με πληροφορίες σχετικά με την εντολή και το probe. Έπειτα δημιουργεί `DTrace probes` για κάθε μία από τις εντολές προς παρακολούθηση. Η εντολή προς παρακολούθηση αποθηκεύεται στην δομή δεδομένων του probe για μετέπειτα χρήση και αντικαθιστάται με μία εντολή `breakpoint` στην συνάρτηση `kinst_enable()`. Μόλις ο CPU κατά την εκτέλεσή του, φτάσει στο `breakpoint` που έχει εγκατασταθεί από το `DTrace`, το `kernel` εισέρχεται στο `DTrace` μέσω του `trap handler` (`sys/$ARCH/$TARGET_ARCH/trap.c`) [25] και εφόσον το `breakpoint` έχει εγκατασταθεί από τον `kinst` συγκεκριμένα, το `kernel` εν τέλει θα εισέλθει στην συνάρτηση `kinst_invp()`. Αυτή η συνάρτηση είναι υπεύθυνη για την παρακολούθηση της εντολής, είτε με την εκτέλεσή της σε «τραμπολίνο» (βλέπε ενότητα 3.4), είτε με την υλοποίηση της σε λογισμικό (`emulation`) (βλέπε ενότητα 3.5).

3.4 Τραμπολίνο (Trampoline)

Τραμπολίνο ονομάζεται ένα εκτελέσιμο κομμάτι μνήμης, το οποίο χρησιμοποιείται ως περιοχή αναπήδησης (`jump`) για την εκτέλεση κάποιας εντολής και επιστροφής στην κανονική ροή του προγράμματος.

Ο `kinst` κάνει χρήση τραμπολίνων για την εκτέλεση των εντολών των οποίων παρακολουθεί, σε αντίθεση με τον `fbt` (2.2), ο οποίος τις «μιμείται»/υλοποιεί σε λογισμικό (`emulation`). Ωστόσο, μία βασική διαφορά του `fbt` είναι ότι εφόσον έχει την δυνατότητα να παρακολουθήσει μόνο την αρχή και το τέλος μίας συνάρτησης, τα οποία σημεία ορίζονται με συγκεκριμένες εντολές `assembly` για κάθε αρχιτεκτονική², το σύνολο των εντολών που πρέπει να υλοποιήσει ο `fbt` είναι μικρό, καθώς και οι εντολές απλές, με αποτέλεσμα να μην χρειάζεται η επιπλέον πολυπλοκότητα που προσθέτει το τραμπολίνο. Ο `kinst` όμως μπορεί να παρακολουθήσει εν δυνάμει (σχεδόν)³ όλες τις εντολές των ISA των αρχιτεκτονικών που υποστηρίζει, οπότε είναι προφανές πως η

²Για παράδειγμα, στην αρχιτεκτονική AMD64, η εντολή `push %rbp` δηλώνει το ξεκίνημα του προλόγου μίας συνάρτησης, και η εντολή `ret` το τέλος του επιλόγου [21].

³Οι εντολές και συναρτήσεις που δεν μπορεί να παρακολουθήσει ο `kinst` αναγράφονται στο manual page του. Ωστόσο το patch που προσθέτει την λίστα με αυτές τις εντολές βρίσκεται σε κατάσταση επιθεώρησης: <https://reviews.freebsd.org/D40874>

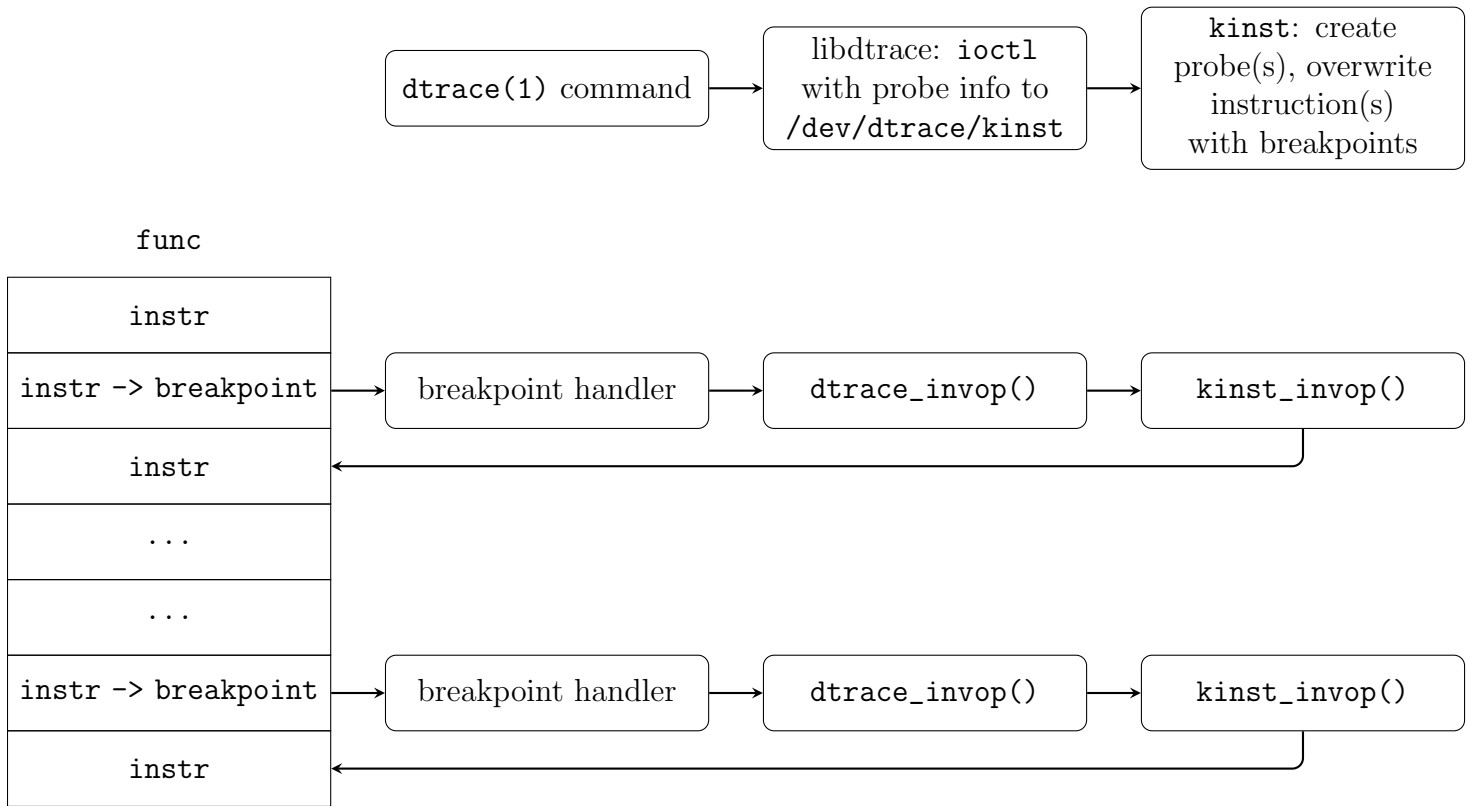


Figure 1: γενική ροή.

προσέγγιση της υλοποίησης εντολών, δεν μπορεί να λειτουργήσει σε αυτήν την περίπτωση. Παρ' όλα αυτά, υπάρχουν μερικές εξαιρέσεις που θα συζητηθούν στην ενότητα 3.5.

Η χρήση του τραμπολίνου γίνεται ως εξής. Αρχικά, όταν το kernel εισέλθει στον **kinst**, η εντολή προς παρακολούθηση αντιγράφεται στο τραμπολίνo, και αλλάζοντας χειροκίνητα την τιμή του PC, η εκτέλεση μεταφέρεται σε αυτό. Αφού εκτελεστεί η αντιγραφμένη εντολή εντός του τραμπολίνου, η εκτέλεση επιστρέφει στην επόμενη εντολή και το kernel επανέρχεται στην κανονική ροή εκτέλεσης.

3.5 Υλοποίηση εντολών (Emulation)

Και στις τρεις αρχιτεκτονικές που υποστηρίζει ο **kinst**, υπάρχουν εντολές, που για λόγους που θα αναλυθούν σε επόμενες ενότητες, δεν μπορούν να εκτελεστούν μέσα σε τραμπολίνo. Αυτές τις εντολές ο **kinst** τις υλοποιεί σε λογισμικό κατά την πυροδότηση του probe. Το θετικό είναι ότι αυτές οι εντολές είναι λίγες και απλές στην υλοποίησή τους.

4 Υλοποίηση

Σε αυτό το κεφάλαιο θα παρουσιαστεί η υλοποίηση του **kinst**, καθώς και της παρακολούθησης inline συναρτήσεων. Επειδή το συνολικό μέγεθος της υλοποίησης είναι χιλιάδες γραμμές κώδικα, στο παρόν έγγραφο θα παρουσιαστούν μόνο τα κομμάτια που είναι ιδιαίτερου ενδιαφέροντος ή/και απαραίτητα για την καλύτερη κατανόηση της υλοποίησης.

Το μεγαλύτερο κομμάτι του πηγαίου κώδικα βρίσκεται στους εξής συνδέσμους: [9] [10] [11]

4.1 kinst

Ο κώδικας του `kinst` βρίσκεται στο `sys/cddl/dev/kinst` [9]:

```
/usr/src/sys/cddl/dev/kinst/
├── aarch64
│   ├── kinst_isa.c
│   └── kinst_isa.h
├── amd64
│   ├── kinst_isa.c
│   └── kinst_isa.h
├── kinst.c
├── kinst.h
├── riscv
│   ├── kinst_isa.c
│   └── kinst_isa.h
└── trampoline.c
```

4.1.1 kinst.*

Στο αρχείο `kinst.c` ορίζονται οι βασικές μέθοδοι φόρτωσης και αποφόρτωσης του kernel module, καθώς και επικοινωνίας του `kinst` με το userspace μέσω της ψευδοσυσκευής `/dev/dtrace/kinst`, αλλά και με το DTrace.

Η φόρτωση και αποφόρτωση του kernel module γίνεται με τα macros `SYSINIT(9)` και `SYSUNINIT(9)` στα οποία δίνονται ως όρισμα τα συναρτήσεις `kinst_load()` και `kinst_unload()` αντίστοιχα, οι οποίες είναι υπεύθυνες για τα εξής:

- Αρχικοποίηση και καθαρισμό του DTrace provider.
- Δημιουργία και καταστροφή της ψευδοσυσκευής.
- Δημιουργία και καταστροφή του τραμπολίνου.
- Αρχικοποίηση και καθαρισμό μερικών παραμέτρων σχετικών με την αρχιτεκτονική στην οποία εκτελείται ο `kinst`.
- Δέσμευση και αποδέσμευση μνήμης για το hashtable με τα probes.
- Εγκατάσταση και απεγκατάσταση της συνάρτησης `kinst_invp()` στο DTrace, η οποία πραγματοποιεί την παρακολούθηση.

Έπειτα, υλοποιούνται οι μέθοδοι της ψευδοσυσκευής. Στο FreeBSD, αυτό γίνεται με την υλοποίηση των function pointers της δομής `cdevsw`, την οποία δώσαμε ως όρισμα κατά την δημιουργία της ψευδοσυσκευής, στην `kinst_load()`. Οι κύριες μέθοδοι που υλοποιούνται συνήθως αντιστοιχούν σε syscalls. Στην προκειμένη περίπτωση, τα syscalls που θέλουμε να υλοποιήσουμε είναι τα `open(2)`, `close(2)`, αλλά κυρίως το `ioctl(2)`, στο οποίο πραγματοποιείται η επικοινωνία του `kinst` με την `libdtrace`, από την οποία λαμβάνει τα απαραίτητα δεδομένα για την δημιουργία του κάθε probe, όπως είδαμε στο σχήμα 1. Η συνάρτηση `kinst_ioctl()`, αφού λάβει το όνομα της συνάρτησης και το/τα offset/s προς παρακολούθηση, ψάχνει στα αρχεία linker του πυρήνα και των φορτωμένων kernel modules για το σύμβολο που αντιστοιχεί στην συνάρτηση που ζητήθηκε. Αν την βρει, καλεί την συνάρτηση `kinst_make_probe()`, η οποία υλοποιείται από κάθε αρχιτεκτονική ξεχωριστά, και είναι υπεύθυνη για την ανάλυση του disassembly της συνάρτησης, την εύρεση της εντολής, και προφανώς την δημιουργία probes.

Τέλος, υλοποιούνται και οι συναρτήσεις `kinst_enable()` και `kinst_disable()`, υπεύθυνες για την ενεργοποίηση και απενεργοποίηση των probes. Αυτές οι συναρτήσεις καλούνται από το DTrace.

4.1.2 trampoline.c

Σε αυτό το αρχείο βρίσκεται η υλοποίηση του τραμπολίνου (βλέπε 3.4).

Ο `kinst` κρατάει μία ουρά `TAILQ(9)` από «χομμάτια» (chunks) μνήμης μεγέθους `PAGE_SIZE`, διαιρεμένα σε επιμέρους τραμπολίνια. Το μέγεθος των τραμπολίνων ορίζεται ξεχωριστά για κάθε αρχιτεκτονική, ανάλογα με τις ανάγκες της. Η μνήμη για κάθε chunk δεσμεύεται με την συνάρτηση `vm_map_find(9)` με την παράμετρο `VM_PROT_EXECUTE`, ώστε να επιτρέπεται η εκτέλεση εντολών εντός του τραμπολίνου. Όλες οι δεσμεύσεις γίνονται σε διευθύνσεις μεγαλύτερες της διεύθυνσης `KERNBASE` στην αρχιτεκτονική AMD64 και της `VM_MIN_KERNEL_ADDRESS` στις υπόλοιπες αρχιτεκτονικές αντίστοιχα.

Το τραμπολίνιο περιέχει την εντολή προς παρακολούθηση, ακολουθούμενη από την εντολή `jmp` στην AMD64, και `breakpoint` στις αρχιτεκτονικές ARM64 και RISC-V αντίστοιχα. Η δεύτερη εντολή χρησιμοποιείται για την επιστροφή από το τραμπολίνιο και την συνέχιση της ροής εκτέλεσης (βλέπε ενότητες 4.1.3 και 4.1.4).

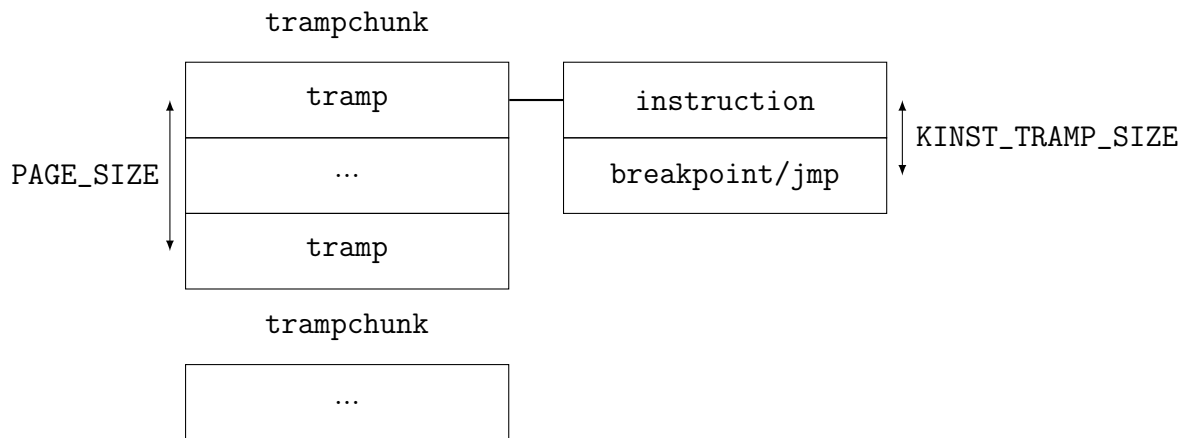


Figure 2: Διάταξη τραμπολίνου.

Όπως αναφέρθηκε προηγουμένως, κατά την φόρτωση του `kinst` δημιουργείται και το τραμπολίνιο μέσω της συνάρτησης `kinst_trampoline_init()`. Η συνάρτηση αυτή λειτουργεί διαφορετικά για την αρχιτεκτονική AMD64 και τις υπόλοιπες — ο λόγος εξηγείται στις ενότητες 4.1.3 και 4.1.4. Πιο συγκεκριμένα, για την AMD64, η `kinst_trampoline_init()` δημιουργεί τραμπολίνια για κάθε thread κάθε διεργασίας. Για τις υπόλοιπες αρχιτεκτονικές, απλώς αρχικοποιεί την λίστα των chunks, εφόσον το κάθε probe έχει το δικό του τραμπολίνιο, οπότε και δημιουργούνται και καταστρέφονται μαζί με το probe.

4.1.3 amd64/kinst_isa.*

Στην ενότητα 4.1.1 έγινε αναφορά στην συνάρτηση `kinst_make_probe()` και στο ποιά είναι η λειτουργία της. Αφού προηγουμένως έχει βρεθεί ότι η συνάρτηση πράγματι υπάρχει στον πυρήνα ή σε kernel module, είναι ώρα για την `kinst_make_probe()` να αναλύσει το disassembly της συνάρτησης — το οποίο λαμβάνει ως όρισμα — και να δημιουργήσει probe(s), αφού ωστόσο έχει αποφασίσει ότι η συγκεκριμένη εντολή (αλλά και η συνάρτηση γενικότερα) είναι εφικτό να παρακολουθηθεί.

Αρχικά, το πρώτο πράγμα που κάνει είναι να δει αν η συνάρτηση βρίσκεται στην λίστα με τις συναρτήσεις που δεν μπορούν να παρακολουθηθούν. Οι συναρτήσεις αυτές συνήθως αντιστοιχούν σε exception handlers χαμηλού επιπέδου, οπότε αποφεύγουμε την οποιαδήποτε παρεμβολή σε αυτές. Έπειτα ξεκινάει η ανάλυση του disassembly. Μία προϋπόθεση που πρέπει να καλυφθεί είναι η συνάρτηση οπωσδήποτε να ξεκινάει με τον κλασικό πρόλογο συνάρτησης, δηλαδή με την εντολή `pushl %rbp`, ειδάλλως η συνάρτηση μπορεί να είναι κάποιος exception handler. Έπειτα, ψάχνει να βρει σε ποιά εντολή αντιστοιχεί το offset προς παρακολούθηση. Αν η εντολή μπορεί να ενεργοποιήσει interrupts (δηλαδή οι `STI` και `POPF`), τότε την απορρίπτουμε διότι η

παρακολούθησή της μπορεί να προκαλέσει προβλήματα. Σε κάθε άλλη περίπτωση, προχωράμε και δημιουργούμε probe για την εντολή που βρέθηκε, ωστόσο χρειάζεται επιπλέον ανάλυση του disassembly.

Στο δεύτερο στάδιο ανάλυσης του disassembly, ψάχνουμε για εντολές που απαιτούν ειδικό χειρισμό κατά την παρακολούθησή τους [16]. Αυτές κατά κύριο λόγο αντιστοιχούν σε εντολές που κωδικοποιούν διευθύνσεις μνήμης συναρτήσεων του καταχωρητή RIP (Instruction Pointer). Ο καταχωρητής αυτός έχει ως τιμή την διεύθυνση μνήμης της επόμενης εντολής από αυτή που εκτελείται σε κάθε δεδομένη στιγμή [20]. Η διευθυνσιοδότηση αυτή ονομάζεται RIP-relative addressing [20]. Ο λόγος που αυτού του είδους η διευθυνσιοδότηση χρειάζεται ειδικό χειρισμό από τον kinst, είναι διότι η τυφλή αντιγραφή μίας τέτοιας εντολής στο τραμπολίνο και η εκτέλεσή της δεν θα λειτουργήσει. Ο λόγος μπορεί να εξηγηθεί με το εξής πείραμα. Ας υποθέσουμε την παρακάτω εντολή:

```
movl $0x01, 0x02(%rip)
```

Υποθέτοντας ότι ο RIP κατά την στιγμή εκτέλεσης της εντολής έχει τιμή 0x100, η ως προς τον RIP διεύθυνση αντιστοιχεί σε 0x02(%rip) → 0x02(0x100) → 0x102, οπότε θεωρητικά έχουμε την παρακάτω εντολή:

```
movl $0x01, [0x102]
```

Αν αντιγράφαμε την αρχική εντολή τυφλά στο τραμπολίνο και προσπαθούσαμε να την εκτελέσουμε, θα καταλήγαμε με μία διαφορετική λάθος διεύθυνση, διότι η τιμή του RIP θα ήταν επίσης διαφορετική. Εφόσον ο RIP έχει ως τιμή την διεύθυνση της επόμενης εντολής από αυτήν που εκτελείται, τότε η τιμή του RIP κατά την εκτέλεση της αντιγραμμένης εντολής στο τραμπολίνο, θα είναι η διεύθυνση της δεύτερης εντολής του τραμπολίνου. Με άλλα λόγια, η διεύθυνση που θα καταλήξει να ορίζει η εντολή δεν θα είναι η επιθυμητή. Οπότε, κατά την αντιγραφή ως προς τον RIP εντολών στο τραμπολίνο, φροντίζουμε να επανακωδικοποιούμε την διεύθυνση συναρτήσεων του τραμπολίνου.

Το τραμπολίνο στην αρχιτεκτονική AMD64 περιέχει την εντολή προς παρακολούθηση, ακολουθούμενη από ένα far-jump στην επόμενη εντολή από αυτή που παρακολουθείται, προκειμένου να συνεχιστεί η ροή εκτέλεσης του πυρήνα.

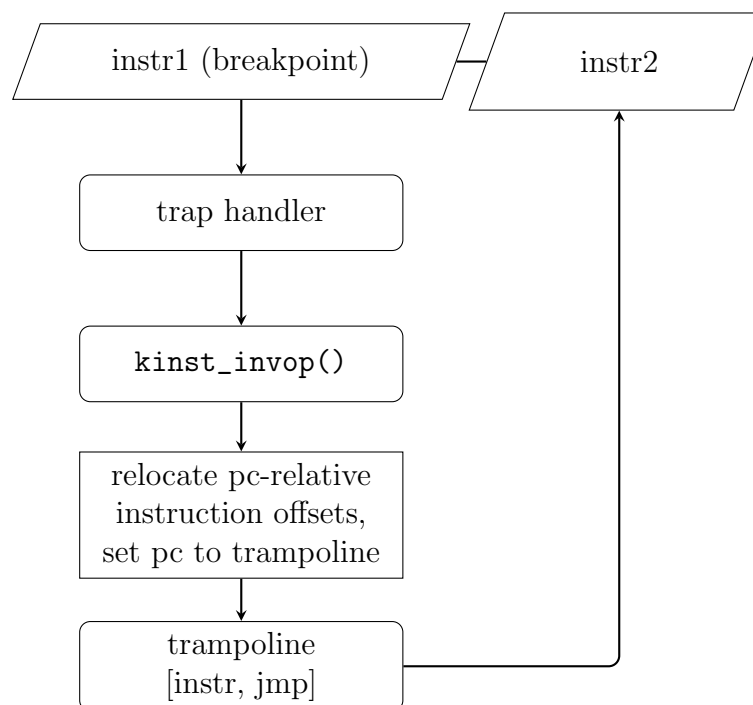


Figure 3: Διάγραμμα ροής amd64.

Για κάθε thread ⁴ και CPU ορίζεται ένα τραμπολίνο, του οποίου τα περιεχόμενα αντικαθίστονται σε κάθε πυροδότηση probe. Ο λόγος για τον οποίο έχουμε τραμπολίνα και ανά thread, αλλά και ανά CPU, είναι ότι τα τραμπολίνα ανά thread δεν μπορούν να χρησιμοποιηθούν με ασφάλεια όταν το thread που εκτελείται είχε απενεργοποιημένα τα interrupts πριν το breakpoint. Αυτός ο μηχανισμός, αν και αποδοτικός από άποψη διαχείρισης μνήμης, μίας και ο αριθμός των τραμπολίνων είναι πάντα ίσος με τον αριθμό των threads και CPUs που έχει το σύστημα, έχει αποδειχθεί προβληματικός όταν τρέχει σε εικονικές μηχανές με πάνω από ένα vCPU, λόγω των συνεχών διαβασμάτων και επανεγγραφών του τραμπολίνου, κάθε φορά που το kernel εισέρχεται στην `kinst_invp()`. Μελλοντικός στόχος είναι να αντικατασταθούν τα τραμπολίνα ανά thread/CPU με τραμπολίνα ανά probe, όπως γίνεται στις αρχιτεκτονικές ARM64 και RISC-V.

Εξαιρέση σε όλα τα παραπάνω ωστόσο είναι οι εντολές `call`, οι οποίες πρέπει να υλοποιηθούν σε λογισμικό, διότι η επανακωδικοποίηση μίας διεύθυνσης ως προς το τραμπολίνο προϋποθέτει δέσμευση χώρου στο stack (βλέπε `bp_call` στο `sys/cddl/dev/dtrace/amd64/dtrace_asm.S`).

4.1.4 aarch64/kinst_isa.* και riscv/kinst_isa.*

Όπως και στην αρχιτεκτονική AMD64 (βλέπε ενότητα 4.1.3), έτσι και στην ARM64 και RISC-V, η συνάρτηση `kinst_make_probe()`, αφού κάνει όλους τους απαραίτητους ελέγχους και σιγουρεύει ότι η συνάρτηση επιτρέπεται να παρακολουθηθεί, δημιουργεί τα probes που ζητήθηκαν.

Πιο συγκεκριμένα, στην αρχιτεκτονική ARM64, η `kinst_make_probe()` αρχικά σιγουρεύει ότι η συνάρτηση δεν ξεκινάει με την εντολή `NOP`, διότι αυτό υποδεικνύει κάποια ειδική συνάρτηση assembly που ίσως να μην είναι ασφαλής η παρακολούθησή της. Επίσης σιγουρεύει ότι η συνάρτηση περιέχει τον κλασικό πρόλογο [22], δηλαδή είτε μία εντολή `STP` της μορφής `stp xt1, xt2, [xn, <const>]!`, είτε `SUB` της μορφής `sub sp, sp, <const>`. Έπειτα ελέγχει ότι η εντολή που αντιστοιχεί στο offset προς παρακολούθηση, δεν βρίσκεται εντός `LDX/STX` block, διότι τα blocks αυτά ορίζουν ατομικές λειτουργίες [23], οπότε η εγκατάσταση breakpoint εντός του block θα ακυρώσει την ατομικότητα, οπότε και δεν είναι δυνατή η παρακολούθηση εντός αυτών των blocks. Τέλος, ελέγχει αν η εντολή πρέπει να υλοποιηθεί από τον `kinst` (βλέπε πίνακα 5) ή αν μπορεί να αντιγραφεί κανονικά στο τραμπολίνο, και δημιουργεί το probe.

Στην αρχιτεκτονική RISC-V, η `kinst_make_probe()` ελέγχει ότι η συνάρτηση ξεκινάει με τον κλασικό πρόλογο και ότι η εντολή προς παρακολούθηση δεν είναι τύπου `SYSTEM` (ενότητα 2.8 του *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA* [18]). η εντολή που αντιστοιχεί στο offset προς παρακολούθηση, δεν βρίσκεται εντός `LR/SC` block, διότι τα blocks αυτά ορίζουν ατομικές λειτουργίες [23], οπότε η εγκατάσταση breakpoint εντός του block θα ακυρώσει την ατομικότητα, οπότε και δεν είναι δυνατή η παρακολούθηση εντός αυτών των blocks (ενότητα 13.3 του *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA* [18]). Τέλος, ελέγχει αν η εντολή πρέπει να υλοποιηθεί από τον `kinst` (βλέπε πίνακα 5) ή αν μπορεί να αντιγραφεί κανονικά στο τραμπολίνο, και δημιουργεί το probe.

Σε αντίθεση με την αρχιτεκτονική AMD64, στις αρχιτεκτονικές ARM64 και RISC-V δεν είναι εφικτό να κωδικοποιηθεί `far-jump` σε μία μόνο εντολή, όπως και δεν είναι εφικτή η απευθείας αλλαγή τιμών καταχωρητών εντός του τραμπολίνου, με σκοπό την υλοποίηση του `far-jump` σε δύο εντολές. Μία εναλλακτική προσέγγιση για την επιστροφή από το τραμπολίνο σε αυτήν την περίπτωση, είναι η χρήση της εντολής `breakpoint`, αντί για `far-jump`.

Αφού εκτελεστεί η εντολή προς παρακολούθηση στο τραμπολίνο και ο CPU φτάσει στην εντολή `breakpoint` (την δεύτερη εντολή του τραμπολίνου), το kernel θα αναπηδήσει πίσω στον trap handler και θα επανεισέλθει στην `kinst_invp()` (βλέπε ενότητα 3.3). Ο `kinst` κρατάει μία εσωτερική ανά CPU δομή προκειμένου να διαφοροποιεί μεταξύ breakpoints που προκλήθηκαν από το τραμπολίνο, ή από probe που πυροδοτήθηκε. Στην πρώτη περίπτωση, η `kinst_invp()` αποθηκεύει τον τρέχον καταχωρητή κατάστασης (state register), απενεργοποιεί τα interrupts ώστε να μην μπορεί να διακοπεί το thread κατά την διάρκεια της εκτέλεσης του «διπλού breakpoint», καθώς και ορίζει τον program counter στην διεύθυνση του τραμπολίνου. Στην δεύτερη

⁴Με την έννοια "thread" στην προκειμένη περίπτωση, δεν αναφερόμαστε σε threads του επεξεργαστή, αλλά στην δομή `thread` του πυρήνα του FreeBSD, η οποία αναπαριστά ένα runnable context εντός μίας διεργασίας (`struct proc`). Κάθε διεργασία μπορεί να έχει πολλαπλές δομές `thread`.

περίπτωση, δηλαδή όταν έχουμε επιστρέψει από το τραμπολίνιο και πρέπει να συνεχίσει η ροή εκτέλεσης, η `kinst_invop()` επαναφέρει τον καταχωρητή κατάστασης και ενεργοποιεί τα interrupts και χειροκίνητα ορίζει τον program counter στην επόμενη εντολή από αυτή που μόλις παρακολούθηθηκε.

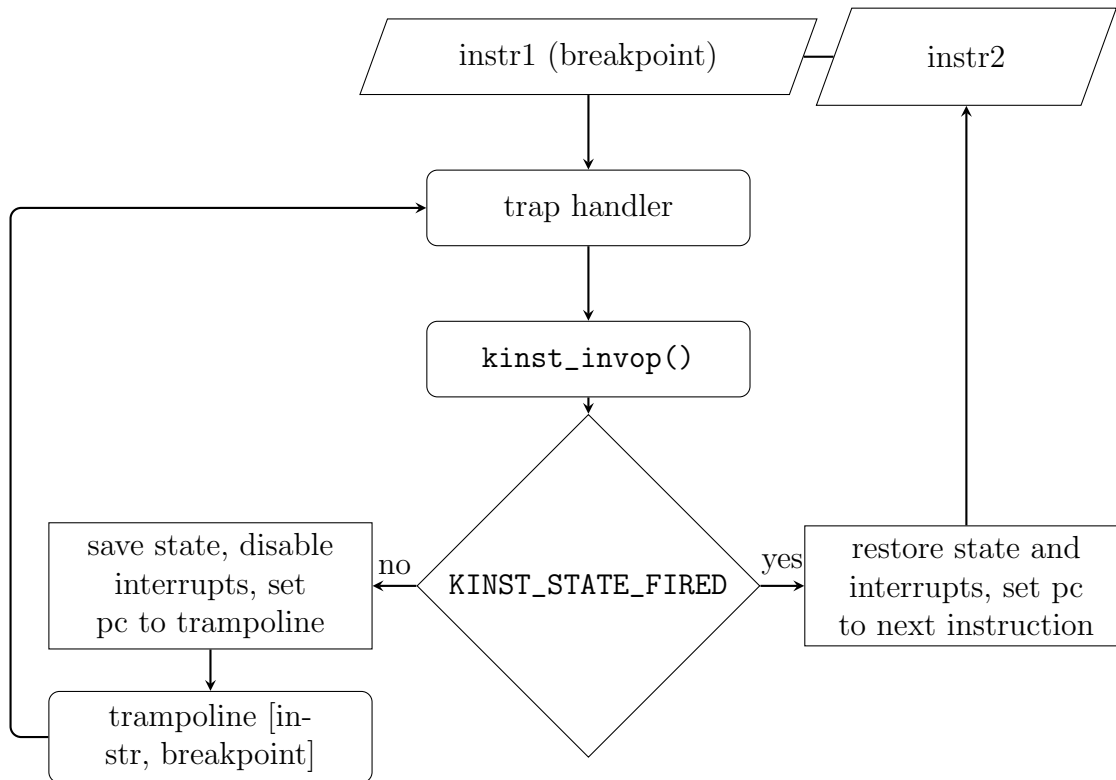


Figure 4: Διάγραμμα ροής arm64 & riscv στην περίπτωση χρήσης τραμπολίνιου.

Κάθε probe έχει το δικό της τραμπολίνιο, το οποίο γράφεται κατά την διάρκεια ανάλυσης της εντολής, το οποίο σε αντίθεση με την αρχιτεκτονική AMD64, αποφεύγει την ανάγκη για διάβασμα και επανεγγραφή τραμπολίνων ανά thread/CPU κάθε φορά που εκτελείται η `kinst_invop()`.

Όπως αναφέρθηκε προηγουμένως, υπάρχουν εντολές που δεν εκτελούνται σε τραμπολίνιο, αλλά υλοποιούνται σε λογισμικό. Ο λόγος και για την αρχιτεκτονική ARM64 και για την RISC-V, είναι ο ίδιος. Σε αντίθεση με την AMD64, όπου για τις εντολές που κωδικοποιούν διευθύνσεις συναρτήσεων του PC, μπορούμε απλώς να επανακωδικοποιήσουμε την διεύθυνση συναρτήσεων του τραμπολίνιου, αυτό δεν μπορεί να γίνει στην ARM64 και RISC-V. Και οι δύο αυτές αρχιτεκτονικές δεν επιτρέπουν την κωδικοποίηση μεγάλων μετατοπίσεων (displacements) σε μία μόνο εντολή, όπως και δεν επιτρέπεται η αλλαγή τιμών καταχωρητών εντός του τραμπολίνιου, με σκοπό την κωδικοποίηση της μετατόπισης σε δύο εντολές αντί για μία. Αυτό έχει ως αποτέλεσμα να μην μπορεί να χρησιμοποιηθεί το τραμπολίνιο για ως προς τον PC εντολές. Για αυτόν τον λόγο οι εντολές αυτές υλοποιούνται από τον `kinst` κατά την εκτέλεση της `kinst_invop()`. Ευτυχώς, οι εντολές είναι απλές οπότε η υλοποίησή τους δεν είναι ιδιαίτερα περίπλοκη διαδικασία. Οι εντολές [19] [17] που υλοποιούνται μέχρι στιγμής από τον `kinst` είναι:

Εντολή	Περιγραφή	Αρχιτεκτονική
adr	Form PC-relative address	ARM64
adrp	From PC-relative address to 4KB page	ARM64
b	Branch	ARM64
b.<condition>	Conditional branch	ARM64
bl	Branch with link	ARM64
cbnz	Compare and branch on non-zero	ARM64
cbz	Compare and branch on zero	ARM64
tbnz	Test bit and branch if non-zero	ARM64
tbz	Test bit and branch if zero	ARM64
jal	Jump and link	RISC-V
jalr	Jump and link register	RISC-V
branch	Branch	RISC-V
auipc	Add upper immediate to PC	RISC-V
c.j	(Compressed) Unconditional jump	RISC-V
c.jr	(Compressed) Jump register	RISC-V
c.jalr	(Compressed) Jump and link register	RISC-V
c.beqz	(Compressed) Branch if zero	RISC-V
c.bnez	(Compressed) Branch if non-zero	RISC-V

Figure 5: Εντολές ARM64 και RISC-V assembly που υλοποιούνται από τον `kinst`.

Και στις δύο αρχιτεκτονικές, οι εντολές υλοποιούνται στις συναρτήσεις `kinst_emulate()`.

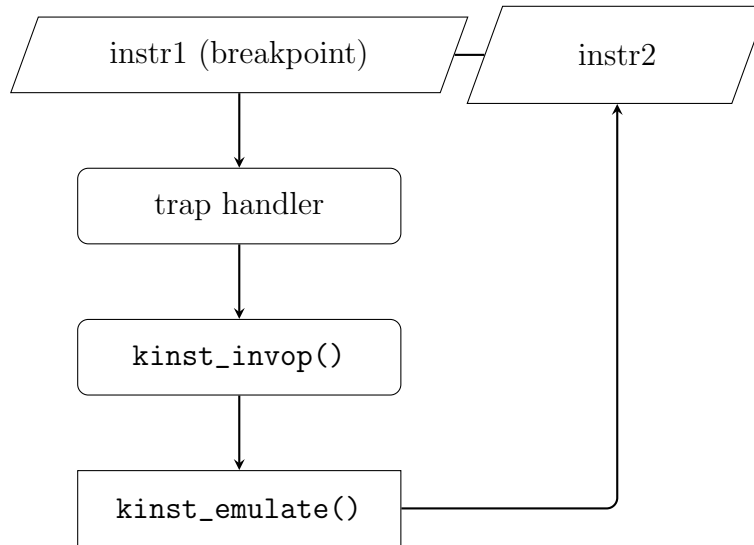


Figure 6: Διάγραμμα ροής arm64 & riscv στην περίπτωση υλοποίησης εντολής.

4.2 Παρακολούθηση inline συναρτήσεων

Όλος ο κώδικας της παρακολούθησης inline συναρτήσεων βρίσκεται στο αρχείο `cddl/contrib/opensolaris/lib/libdtrace/common/dt_sugar.c` [10], το οποίο είναι μέρος του compiler των D scripts.

Το `dt_sugar.c` διαχειρίζεται μία συνδεδεμένη λίστα με όλα τα clauses ενός δεδομένου D script. Για την υλοποίηση της inline παρακολούθησης, εφαρμόζεται πάνω σε αυτήν την συνδεδεμένη λίστα ό,τι θα εξηγηθεί στις επόμενες υποενότητες.

4.2.1 Τι είναι οι inline συναρτήσεις και γιατί είναι δύσκολη η παρακολούθησή τους;

Inline συναρτήσεις είναι συναρτήσεις τις οποίες ο μεταγλωττιστής μετατρέπει σε απλά κομμάτια κώδικα τα οποία εκτελούνται εντός της καλούσας συνάρτησης, με σκοπό την βελτίωση της επίδοσης του κώδικα. Για παράδειγμα, ας υποθέσουμε το παρακάτω:

```
static inline void
foo(void)
{
    printf("Hello world\n");
}

static void
bar(void)
{
    foo();
}
```

Εφόσον η συνάρτηση `foo()` έχει οριστεί ως `inline`⁵, ο μεταγλωττιστής θα μετατρέψει τον κώδικα στην παρακάτω μορφή:

```
static void
bar(void)
{
    printf("Hello world\n");
}
```

Αυτή η βελτιστοποίηση ωστόσο δημιουργεί πρόβλημα για το DTrace στην περίπτωση που θέλουμε να παρακολουθήσουμε την συνάρτηση `foo()`, διότι η συνάρτηση αυτή, παρόλο που έχει οριστεί στον κώδικα, δεν υπάρχει στο τελικό εκτελέσιμο πρόγραμμα, με αποτέλεσμα το DTrace να μην μπορεί να βρει πληροφορίες για το σύμβολο `foo()`. Αυτός είναι ένας από τους μεγάλους περιορισμούς του `fbt`, τον οποίο λύνει ο `kinst`.

4.2.2 Πώς ο `kinst` λύνει αυτό το πρόβλημα

Στην πραγματικότητα, ο `kinst` δεν διαθέτει επιπλέον λογική όσον αφορά τις `inline` συναρτήσεις — η λύση αυτού του προβλήματος βρίσκεται σε προηγούμενο στάδιο, σε αυτό της συντακτικής ανάλυσης του D script από την `libdtrace`.

Η παρακολούθηση `inline` συναρτήσεων στον `kinst` δεν είναι τίποτα παραπάνω από ένα συντακτικό στρώμα που έχει προστεθεί στην `libdtrace` [15]. Όταν ο χρήστης ζητάει ένα `probe` της μορφής `kinst::<function>:<entry|return>`⁶, η `libdtrace` αναλύει τις πληροφορίες DWARF (βλέπε ενότητα 4.2.3) όλων των φορτωμένων `kernel modules`⁷, με σκοπό να βρει αν η συνάρτηση `<function>` είναι `inline`. Σε αυτή την περίπτωση, δημιουργεί `kinst probes` στις συναρτήσεις και ακριβή `offset(s)`, στα οποία έχει βρεθεί `inline` αντίγραφο της `<function>`. Ωστόσο, αν η `<function>` δεν είναι `inline`, το `probe` μετατρέπεται αυτόματα σε `fbt probe`, μίας και ο `fbt` είναι ιδανικός για την παρακολούθηση της αρχής και τέλους `μη-inline` συναρτήσεων. Το σχήμα 7 παρουσιάζει ένα παράδειγμα του πώς η `libdtrace` χειρίζεται `inline` και `μη-inline` συναρτήσεις.

⁵Αν δεν οριστεί ρητά, ο μεταγλωττιστής αν κρίνει κατάλληλο, πολλές φορές μπορεί να μετατρέψει μία συνάρτηση σε `inline` ακόμα και αν αυτή δεν έχει οριστεί ως τέτοια.

⁶Σε αντίθεση με τον `fbt`, ο `kinst` δεν μπορεί να δεχθεί άδαιο το πεδίο `probe` όταν πρόκειται για παρακολούθηση `inline` συναρτήσεων. Προκειμένου να μπορέσει να ενεργοποιηθεί ο κώδικας εντοπισμού `inline` συναρτήσεων εντός του `libdtrace`, πρέπει αναγκαστικά να οριστεί είτε η λέξη `entry` είτε `return`.

⁷Αυτό ωστόσο καθιστά την όλη διαδικασία ιδιαίτερα χρονοβόρα. Μελλοντικός στόχος είναι να μειωθεί όσο το δυνατόν περισσότερο αυτή η επιβάρυνση στην επίδοση.

Inline function

```
kinst::cam_iosched_has_more_trim:entry
{
    printf("\t%d\t%s", pid, execname);
}
```



```
kinst::cam_iosched_get_trim:13,
kinst::cam_iosched_next_bio:13,
kinst::cam_iosched_schedule:40
{
    printf("\t%d\t%s", pid, execname);
}
```

Non-inline function

```
kinst::malloc:entry
{
    exit(0);
}
```



```
fbt::malloc:entry
{
    exit(0);
}
```

Figure 7: Συντακτικοί μετασχηματισμοί υλοποιημένοι στο `cddl/contrib/opensolaris/lib/libdtrace/common/dt_sugar.c`

Υπάρχει περίπτωση να υπάρχουν inline αλλά και μη-inline ορισμοί της ίδιας συνάρτησης. Σε αυτήν την περίπτωση, η `libdtrace`, πέρα από τα `kinst` probes για τα inline αντίγραφα, δημιουργεί και ένα επιπλέον `fbt` probe για την μη-inline συνάρτηση.

Για την παραγωγή των εξόδων που παρουσιάζονται στο σχήμα 7, χρησιμοποιείται η επιλογή `-d` στην εντολή `dtrace(1)`, η οποία τυπώνει το D script αφού έχουν εφαρμοστεί όλοι οι συντακτικοί μετασχηματισμοί. Η επιλογή αυτή προστέθηκε με το commit `1e136a9cbd3a`.

Μέχρι και την στιγμή της συγγραφής της παρούσας εργασίας, η υλοποίηση της παρακολούθησης *inline* συναρτήσεων βρίσκεται σε κατάσταση επιθεώρησης [12].

4.2.3 Πρότυπο DWARF

Το πρότυπο αποσφαλμάτωσης DWARF [13] είναι ένα πρότυπο το οποίο χρησιμοποιείται από μεταγλωττιστές και προγράμματα αποσφαλμάτωσης (debuggers) για την κωδικοποίηση χρήσιμων πληροφοριών, όπως ονόματα συναρτήσεων ή θέσεις μεταβλητών, και πολλών άλλων, για κάθε μονάδα μεταγλώττισης (compilation unit).

This document defines a format for describing programs to facilitate user source level debugging. This description can be generated by compilers, assemblers and linkage editors. It can be used by debuggers and other tools.

– The DWARF Standard: <https://dwarfstd.org/>

Οι πληροφορίες αυτές αναπαριστώνται ως ένα δέντρο από εγγραφές, οι οποίες ονομάζονται DIEs. Η κάθε μονάδα μεταγλώττισης αντιστοιχεί σε ένα τέτοιο δέντρο. Για καλύτερη κατανόηση, ας θεωρήσουμε το παρακάτω θεωρητικό δέντρο:

Κατά την μεταγλώττιση ενός C προγράμματος, η ενσωμάτωση των πληροφοριών DWARF εντός του μεταγλωττισμένου αρχείου μπορεί να γίνει με την χρήση της επιλογής `-g` (παραγωγή πληροφοριών αποσφαλμάτωσης):

```
# cc <file>.c -g -o <file>
```

Οι πληροφορίες DWARF μπορούν έπειτα να διαβαστούν είτε με το πρόγραμμα `readelf` (με την επιλογή `-wi`):

```
# readelf -wi <file>
```

Είτε με το `dwarfdump`:

```
# dwarfdump <file>
```

Οι δηλώσεις των inline συναρτήσεων στο πρότυπο DWARF αναπαριστώνται με την ετικέτα `DW_TAG_subprogram`, σε συνδυασμό με την ιδιότητα `DW_AT_inline`. Ωστόσο για κάθε δήλωση μίας inline συνάρτησης, υπάρχουν και αντίγραφά της, ένα για κάθε σημείο στο οποίο η συνάρτηση έχει μετατραπεί σε inline εντός μίας άλλης συνάρτησης. Τα αντίγραφα αυτά αναπαριστώνται με την ετικέτα `DW_TAG_inlined_subroutine` και μπορούν να περιέχουν τις παρακάτω ιδιότητες:

- `DW_AT_abstract_origin`: Θέση του DIE που περιέχει την δήλωση της inline συνάρτησης.
- `DW_AT_call_file`: Ακέραιος αριθμός που δηλώνει το αρχείο στο οποίο καλείται η συνάρτηση.
- `DW_AT_call_line`: Γραμμή κώδικα εντός του αρχείου, στο οποίο καλείται η συνάρτηση.
- `DW_AT_call_column`: Στήλη εντός της γραμμής, όπου ξεκινάει η κλήση της συνάρτησης.
- `DW_AT_low_pc` και `DW_AT_high_pc`: Κάτω και άνω όρια (διευθύνσεις μνήμης) εντός των οποίων βρίσκεται το αντίγραφο.
- `DW_AT_ranges`: Σειρές ορίων εντός των οποίων βρίσκεται το αντίγραφο, σε περίπτωση που το αντίγραφο δεν βρίσκεται σε ενιαίες θέσεις μνήμης.

Όσον αφορά τις inline συναρτήσεις, ας υποθέσουμε την παρακάτω εντολή `DTrace`. Όλη η διαδικασία που παρουσιάζεται παρακάτω γίνεται προγραμματιστικά εντός της `libdtrace` με την χρήση της βιβλιοθήκης DWARF (`dwarf(3)`) [14], ωστόσο για χάρην κατανόησης θα παρουσιαστεί χειροκίνητα:

```
# dtrace -n 'kinst::vfs_freevnodes_dec:entry'
```

Εντοπίζοντας την συνάρτηση στον κώδικα του FreeBSD (`sys/kern/vfs_subr.c`):

```
static __inline void
vfs_freevnodes_dec(void)
{
    [...]
}
```

Εφόσον η συνάρτηση `vfs_freevnodes_dec()` είναι μέρος του FreeBSD kernel, αλλά και inline, η `libdtrace` πρέπει αρχικά να αναλύσει τα δεδομένα DWARF του kernel, τα οποία βρίσκονται στο αρχείο `/usr/lib/debug/boot/kernel/kernel.debug`, να βρει αν η `vfs_freevnodes_dec()` είναι inline (που στην προκειμένη περίπτωση, είναι), και έπειτα να βρει όλα τα inline αντίγραφα της και να δημιουργήσει `kinst probes` στα σημεία που εντοπίστηκαν. Αν και όλα αυτά γίνονται εντός της `libdtrace`, χάριν κατανόησης και απεικόνισης της διαδικασίας που ακολουθείται, θα χρησιμοποιήσουμε την παρακάτω εντολή για να εμφανίσουμε τα δεδομένα DWARF του kernel:

```
# readelf -wi /usr/lib/debug/boot/kernel/kernel.debug | less
```

Και έπειτα θα ψάξουμε για το παρακάτω DIE, το οποίο αντιστοιχεί στην συνάρτηση `vfs_freevnodes_dec()`:

```

<1><1f03fc9>: Abbrev Number: 99 (DW_TAG_subprogram)
  <1f03fca>   DW_AT_name           : (indirect string) vfs_freevnodes_dec
  <1f03fce>   DW_AT_decl_file      : 1
  <1f03fcf>   DW_AT_decl_line     : 1584
  <1f03fd1>   DW_AT_prototyped    : 1
  <1f03fd1>   DW_AT_inline        : 1

```

Listing 1: DIE δήλωσης inline συνάρτησης

Η ετικέτα `DW_TAG_subprogram` υποδηλώνει ότι το DIE αναφέρεται σε συνάρτηση, και η ιδιότητα `DW_AT_inline` ότι η συνάρτηση είναι inlined. Τώρα που η `libdtrace` ξέρει ότι η συνάρτηση είναι πράγματι inlined, πρέπει να βρει τα DIE που αντιστοιχούν στο κάθε inline αντίγραφο. Αυτό γίνεται αναζητώντας DIEs τα οποία ορίζουν ως `DW_AT_abstract_origin` το offset της δήλωσης της συνάρτησης. Στην προκειμένη περίπτωση αυτό το offset είναι το `0x1f03fc9` (βλέπε πρώτη σειρά στο Listing 1). Για παράδειγμα:

```

<3><1f09100>: Abbrev Number: 24 (DW_TAG_inlined_subroutine)
  <1f09101>   DW_AT_abstract_origin: <0x1f03fc9>
  <1f09105>   DW_AT_low_pc         : 0xffffffff80c60be1
  <1f0910d>   DW_AT_high_pc        : 0x4e
  <1f09111>   DW_AT_call_file      : 1
  <1f09112>   DW_AT_call_line     : 3744
  <1f09114>   DW_AT_call_column   : 3

```

Listing 2: DIE αντιγράφου inline συνάρτησης

Όπως αναφέρθηκε παραπάνω, τα inline αντίγραφα μίας συνάρτησης υποδηλώνονται με την ετικέτα `DW_TAG_inlined_subroutine`. Η ιδιότητα `DW_AT_abstract_origin` ορίζει το offset στο οποίο βρίσκεται η δήλωση της δήλωσης της συνάρτησης, σε αυτή την περίπτωση `0x1f03fc9`, το οποίο αντιστοιχεί στο DIE της δήλωσης της `vfs_freevnodes_dec()` (βλέπε Listing 1).

4.2.4 Υπολογισμός ορίων κλήσεων inline αντιγράφων

Το DWARF καθορίζει τα όρια κλήσεων συναρτήσεων με έναν από τους δύο τρόπους για κάθε DIE:

- Ορίζοντας τις ιδιότητες `DW_AT_low_pc` και `DW_AT_high_pc`.
- Ορίζοντας την ιδιότητα `DW_AT_ranges` όταν το εύρος διευθύνσεων του inline αντιγράφου δεν είναι συνεχόμενο.

Στην πρώτη περίπτωση, το κάτω όριο της θέσης του inline αντιγράφου ορίζεται στο `DW_AT_low_pc` και το άνω όριο υπολογίζεται προσθέτοντας το `DW_AT_high_pc` στο `DW_AT_low_pc`. Χρησιμοποιώντας ως παράδειγμα την συνάρτηση `vfs_freevnodes_dec()`, ως συνέχεια της ενότητας 4.2.3:

```

<3><1f09100>: Abbrev Number: 24 (DW_TAG_inlined_subroutine)
  <1f09101>   DW_AT_abstract_origin: <0x1f03fc9>
  <1f09105>   DW_AT_low_pc         : 0xffffffff80c60be1
  <1f0910d>   DW_AT_high_pc        : 0x4e
  <1f09111>   DW_AT_call_file      : 1
  <1f09112>   DW_AT_call_line     : 3744
  <1f09114>   DW_AT_call_column   : 3

```

Listing 3: DIE με κάτω και άνω όρια PC

Χρησιμοποιώντας τον τύπο που αναφέρθηκε στην προηγούμενη παράγραφο, έχουμε τα παρακάτω όρια:

$$\begin{aligned} Bound_{low} &= PC_{low} = 0xffffffff80c60be1 \\ Bound_{high} &= PC_{low} + PC_{high} = 0xffffffff80c60be1 + 0x4e = \\ &0xffffffff80c60c2f \end{aligned} \quad (1)$$

Η δεύτερη περίπτωση είναι λίγο πιο περίπλοκη. Ας υποθέσουμε το παρακάτω DIE ενός inline αντιγράφου της `vfs_freevnodes_dec()` που έχει την ιδιότητα `DW_AT_ranges`:

```
<3><1f0774a>: Abbrev Number: 28 (DW_TAG_inlined_subroutine)
  <1f0774b>   DW_AT_abstract_origin: <0x1f03fc9>
  <1f0774f>   DW_AT_ranges          : 0x203730
  <1f07753>   DW_AT_call_file       : 1
  <1f07754>   DW_AT_call_line      : 3744
  <1f07756>   DW_AT_call_column    : 3
```

Listing 4: DIE με όρια `DW_AT_ranges`

Η ιδιότητα `DW_AT_ranges` αναφέρεται στην ενότητα `.debug_ranges` που βρίσκεται εντός των debug αρχείων και μπορεί να εμφανισθεί τρέχοντας την εντολή `dwarfdump -N <file>`. Αναζητώντας για την παραπάνω τιμή `DW_AT_ranges 0x203730`, βρίσκουμε την αντίστοιχη ομάδα εύρους, τρέχοντας την εντολή `dwarfdump -N /usr/lib/debug/boot/kernel/kernel.debug`:

```
Ranges group 40228:
  ranges: 3 at .debug_ranges offset 2111280 (0x00203730) (48 bytes)
    [ 0] range entry    0x0000271b 0x00002754
    [ 1] range entry    0x000027ca 0x000027e5
    [ 2] range end      0x00000000 0x00000000
```

Listing 5: Παράδειγμα `DW_AT_ranges`

Όλες οι γραμμές «range entry» αντιστοιχούν σε διαφορετικά εύρη διευθύνσεων στα οποία έχει χωριστεί το inline αντίγραφο, συνήθως ως αποτέλεσμα πρόωρων return εντός της συνάρτησης. Τα όρια κλήσεων υπολογίζονται προσθέτοντας κάθε τιμή των «range entries» στο κάτω όριο (`DW_AT_low_pc`) του root DIE, δηλαδή του DIE ανώτατου επιπέδου, που αντιστοιχεί στο αρχείο εντός του οποίου είναι υλοποιημένη η inline συνάρτηση. Ένας εύκολος τρόπος να βρεθεί χειροκίνητα το DIE είναι με την αναζήτηση του ονόματος του αρχείου που είναι υλοποιημένη συνάρτηση, στην προκειμένη περίπτωση `sys/kern/vfs_subr.c`:

```
<0><1ef6226>: Abbrev Number: 1 (DW_TAG_compile_unit)
  <1ef6227>   DW_AT_producer       : (indirect string) FreeBSD clang version 19.1.7 (
https://github.com/llvm/llvm-project.git llvmorg-19.1.7-0-gcd708029e0b
2)
  <1ef622b>   DW_AT_language       : 29 (C11)
  <1ef622d>   DW_AT_name           : (indirect string) /usr/src/sys/kern/vfs_subr.c
  <1ef6231>   DW_AT_stmt_list      : 0x6946a0
  <1ef6235>   DW_AT_comp_dir       : (indirect string) /usr/obj/usr/src/amd64.amd64/sys/
GENERIC
  <1ef6239>   DW_AT_low_pc         : 0xffffffff80c5d810
  <1ef6241>   DW_AT_high_pc        : 0xe9a3
```

Listing 6: DIE μονάδας μεταγλώττισης

Προσθέτοντας τα range entries στο κάτω όριο του root DIE (`0xffffffff80c5d810`), έχουμε τα παρακάτω όρια κλήσης του inline αντιγράφου:

Πρώτο range entry:

$$\begin{aligned} Bound0_{low} &= Root_{lowpc} + Range0_{low} = 0xffffffff80c5d810 + 0x0000271b = \\ &0xffffffff80c5ff2b \\ Bound0_{high} &= Root_{lowpc} + Range0_{high} = 0xffffffff80c5d810 + 0x00002754 = \\ &0xffffffff80c5ff64 \end{aligned} \quad (2)$$

Δεύτερο range entry:

$$\begin{aligned} Bound1_{low} &= Root_{lowpc} + Range1_{low} = 0xffffffff80c5d810 + 0x000027ca = \\ &0xffffffff80c5ffda \\ Bound1_{high} &= Root_{lowpc} + Range1_{high} = 0xffffffff80c5d810 + 0x000027e5 = \\ &0xffffffff80c5fff5 \end{aligned} \quad (3)$$

Το $Bound0_{low}$ είναι το σημείο αρχής της συνάρτησης (entry), και τα $Bound0_{high}$ και $Bound1_{high}$ τα σημεία επιστροφής (return).

4.2.5 Εύρεση καλούσας συνάρτησης

Αφού έχουν υπολογισθεί τα όρια κλήσεων των inline αντιγράφων, η libdtrace μπορεί να προχωρήσει στην εύρεση των ονομάτων των καλούντων συναρτήσεων αυτών των αντιγράφων. Τα ονόματα αυτά χρειάζονται για την δημιουργία kinst probes. Αυτό γίνεται βρίσκοντας εντός ποιού συμβόλου ELF βρίσκονται τα όρια κλήσεων, όπως εκφράζεται από την παρακάτω συνθήκη:

$$Sym_{low} \leq Inline_{low} \leq Inline_{high} \leq Sym_{high}$$

Το πρότυπο ELF [24] προήλθε από τα Unix System Laboratories και ορίζει μία τυποποιημένη μορφή εκτελέσιμων αρχείων, αντικειμενικού κώδικα (object code), μοιρασμένων βιβλιοθηκών (shared libraries) και core dumps. Είναι το καθιερωμένο πρότυπο πολλών λειτουργικών συστημάτων, ειδικά αυτών που προέρχονται από ή είναι παρεμφερή με το UNIX.

Όπως και στην περίπτωση της ανάλυσης των δεδομένων DWARF, έτσι και εδώ η διαδικασία γίνεται όλη προγραμματιστικά εντός της libdtrace, με την χρήση της βιβλιοθήκης ELF (elf(3)). Ωστόσο, και εδώ για χάρην κατανόησης θα παρουσιάσουμε την διαδικασία χειροκίνητα.

Για την εμφάνιση των συμβόλων ELF, μπορούμε να εκτελέσουμε την εντολή `readelf -s` στο εκτέλεσιμο αρχείο του kernel:

```
# readelf -s /usr/lib/debug/boot/kernel/kernel.debug
```

Η παραπάνω εντολή τυπώνει πληροφορίες για όλα τα σύμβολα εντός ενός εκτελέσιμου αρχείου σε μορφή πίνακα:

```
# readelf -s /usr/lib/debug/boot/kernel/kernel.debug | head
```

Symbol table '.symtab' contains 71137 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	ffffffff80387100	0	NOTYPE	LOCAL	DEFAULT	9	1st
2:	ffffffff80387087	0	NOTYPE	LOCAL	DEFAULT	9	11
3:	ffffffff803870d1	0	NOTYPE	LOCAL	DEFAULT	9	12
4:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	cam.c
5:	ffffffff80387600	10	FUNC	LOCAL	DEFAULT	9	camstatusentrycomp
6:	ffffffff81804000	112	OBJECT	LOCAL	DEFAULT	50	sysctl___kern_features_scbus

Στην τέταρτη στήλη αναγράφεται ο τύπος του συμβόλου. Εφόσον έχουμε να κάνουμε μόνο με συναρτήσεις, ο τύπος που μάς ενδιαφέρει είναι ο τύπος **FUNC**, οπότε η έξοδος θα φιλτραριστεί ανάλογα. Η δεύτερη στήλη ορίζει την διεύθυνση μνήμης (σε δεκαεξαδικό) στην οποία είναι τοποθετημένο το κάθε σύμβολο, και η τρίτη στήλη το μέγεθος του συμβόλου (σε δεκαδικό). Με αυτές τις δύο στήλες μπορούμε να βρούμε τα όρια κάθε συμβόλου.

Ας υποθέσουμε μία τυχαία γραμμή που ορίζει σύμβολο τύπου **FUNC**, δηλαδή μία συνάρτηση:

15143: ffffffff80fc7540	4286 FUNC	GLOBAL DEFAULT	9 pmap_enter
-------------------------	-----------	----------------	--------------

Παρατηρούμε στην δεύτερη στήλη ότι η διεύθυνση μνήμης στην οποία είναι τοποθετημένη η συνάρτηση **pmap_enter()** είναι η **0xffffffff80fc7540**. Με άλλα λόγια αυτό είναι και το κάτω όριο της συνάρτησης. Στην τρίτη στήλη ορίζεται το μέγεθος της, δηλαδή 4286, οπότε προστίθοντας την τιμή αυτή στο κάτω όριο, μπορούμε να υπολογίσουμε και το άνω όριο της συνάρτησης:

$$Sym_{low} = 0xffffffff80fc7540$$

$$Sym_{high} = 0xffffffff80fc7540 + 4286 = 0xffffffff80fc85fe$$

Με αυτά τα δεδομένα, σε συνδυασμό με τα όρια των **inline** αντιγράφων που υπολογίστηκαν στην ενότητα 4.2.4, μπορούμε να βρούμε τα ονόματα των καλούσων συναρτήσεων, καθώς και τα ακριβή **offsets** στα οποία εμφανίζονται τα **inline** αντίγραφα.

Το παρακάτω **Lua script** δέχεται ως είσοδο την έξοδο της εντολής **readelf -s** και υλοποιεί τον μηχανισμό που αναλύθηκε σε αυτήν την ενότητα. Δεν είναι μέρος της υλοποίησης, αλλά απλώς ένα **script** επίδειξης του μηχανισμού επαλήθευσής της υλοποίησης:

```
#!/usr/libexec/flua

-- Usage: caller.lua <inline_low> <inline_high>

-- Read the inline function boundaries from the command line arguments
local inl_lo = tonumber(arg[1])
local inl_hi = tonumber(arg[2])

-- Split string by line
local function magiclines(s)
    if s:sub(-1)~="\n" then s=s.."\n" end
    return s:gmatch("(.-)\n")
end

-- Read stdin line by line
for line in magiclines(io.read("*a")) do
    local cols = {}
    local sym_lo = {}
    local sym_hi = {}

    -- Only match FUNC (i.e., function) symbols
    if string.find(line, "FUNC") then
        -- Split each line into columns
        for s in string.gmatch(line, "([^\s]+)") do
            cols[#cols + 1] = s
        end
    end
end
```



```

-- Calculate the symbol's lower and upper boundaries
sym_lo = tonumber(cols[2], 16)
sym_hi = tonumber(cols[2], 16) + tonumber(cols[3], 10)

-- See if the inline function is inlined within current symbol
if sym_lo <= inl_lo and inl_hi <= sym_hi then
    print(cols[#cols])
    print("\tsym_lo=\t", string.format("0x%x", sym_lo))
    print("\tsym_hi=\t", string.format("0x%x", sym_hi))
    print("\tinl_lo=\t", string.format("0x%x", inl_lo))
    print("\tinl_hi=\t", string.format("0x%x", inl_hi))
end
end
end

```

Για παράδειγμα, χρησιμοποιώντας τα όρια ενός εκ των inline αντιγράφων της `vfs_freevnodes_dec()`, που υπολογίστηκαν στην εξίσωση 1, μπορούμε να βρούμε εντός ποιās συνάρτησης έχει μετατραπεί σε inline:

```

# readelf -s /usr/lib/debug/boot/kernel/kernel.debug | ./caller.lua 0xffffffff80c60be1
0xffffffff80c60c2f
vget_prep
    sym_lo=          0xffffffff80c60ba0
    sym_hi=          0xffffffff80c60c36
    inl_lo=          0xffffffff80c60be1
    inl_hi=          0xffffffff80c60c2f

```

Οπότε το inline αντίγραφο της `vfs_freevnodes_dec()` με όρια `[0xffffffff80c60be1, 0xffffffff80c60c2f]` έχει μετατραπεί σε inline εντός της συνάρτησης `vget_prep()`. Το script τυπώνει επιπλέον και τα όρια της καλούσας συνάρτησης και του inline αντιγράφου, τα οποία θα χρησιμεύσουν στην επόμενη ενότητα (4.2.6).

4.2.6 Υπολογισμός των offsets entry και return

Αυτό είναι το τελευταίο βήμα που πρέπει να ακολουθήσει η `libdtrace` προκειμένου να μετατρέψει ένα probe inline συνάρτησης σε `kinst probe(s)` στα ακριβή offsets (`entry` ή `return`) που εμφανίζεται κάθε inline αντίγραφο μίας inline συνάρτησης. Στην ενότητα 4.2.5 παρουσιάστηκε ο μηχανισμός με τον οποίο βρίσκεται το όνομα της καλούσας συνάρτησης μίας inline συνάρτησης, και έγινε επίδειξη του μηχανισμού μέσω ενός Lua script, το οποίο τυπώνει το όνομα της καλούσας συνάρτησης, καθώς και τα όρια της και του inline αντιγράφου. Με αυτά τα δεδομένα μπορούν να υπολογιστούν τα `entry` και `return` offsets.

Αρχικά, παρατίθεται το `disassembly` της συνάρτησης `vget_prep()`, στην οποία εμφανίζεται το inline αντίγραφο που εξετάσαμε στην ενότητα 4.2.5, διότι οι πληροφορίες αυτές θα είναι χρήσιμες:

```

(gdb) disas vget_prep
Dump of assembler code for function vget_prep:
0xffffffff80c60ba0 <+0>:    push    %rbp
0xffffffff80c60ba1 <+1>:    mov     %rsp,%rbp
0xffffffff80c60ba4 <+4>:    mov     0x1a4(%rdi),%eax
0xffffffff80c60baa <+10>:   test    %eax,%eax
0xffffffff80c60bac <+12>:   jle     0xffffffff80c60bc2 <vget_prep+34>
0xffffffff80c60bae <+14>:   lea     0x1(%rax),%ecx
0xffffffff80c60bb1 <+17>:   lock   cmpxchg %ecx,0x1a4(%rdi)
0xffffffff80c60bb9 <+25>:   jne     0xffffffff80c60baa <vget_prep+10>

```

```

0xffffffff80c60bbb <+27>:  mov    $0x2,%eax
0xffffffff80c60bc0 <+32>:  pop    %rbp
0xffffffff80c60bc1 <+33>:  ret
0xffffffff80c60bc2 <+34>:  mov    $0x2,%eax
0xffffffff80c60bc7 <+39>:  je     0xffffffff80c60bcb <vget_prep+43>
0xffffffff80c60bc9 <+41>:  pop    %rbp
0xffffffff80c60bca <+42>:  ret
0xffffffff80c60bcb <+43>:  mov    $0x1,%eax
0xffffffff80c60bd0 <+48>:  mov    $0x1,%ecx
0xffffffff80c60bd5 <+53>:  lock  xadd %ecx,0x1a0(%rdi)
0xffffffff80c60bdd <+61>:  test   %ecx,%ecx
0xffffffff80c60bdf <+63>:  jne    0xffffffff80c60bc9 <vget_prep+41>
0xffffffff80c60be1 <+65>:  mov    %gs:0x0,%rcx
0xffffffff80c60bea <+74>:  incl   0x4fc(%rcx)
0xffffffff80c60bf0 <+80>:  mov    %gs:0x180,%rdi
0xffffffff80c60bf9 <+89>:  movzbl 0x88(%rdi),%edx
0xffffffff80c60c00 <+96>:  dec    %dl
0xffffffff80c60c02 <+98>:  mov    %dl,0x88(%rdi)
0xffffffff80c60c08 <+104>: cmp     $0x82,%dl
0xffffffff80c60c0b <+107>: je     0xffffffff80c60c23 <vget_prep+131>
0xffffffff80c60c0d <+109>: decl   0x4fc(%rcx)
0xffffffff80c60c13 <+115>: cmpb   $0x0,0x138(%rcx)
0xffffffff80c60c1a <+122>: je     0xffffffff80c60bc9 <vget_prep+41>
0xffffffff80c60c1c <+124>: call   0xffffffff80b7ce70 <critical_exit_preempt>
0xffffffff80c60c21 <+129>: jmp    0xffffffff80c60c2f <vget_prep+143>
0xffffffff80c60c23 <+131>: add    $0x88,%rdi
0xffffffff80c60c2a <+138>: call   0xffffffff80c6a960 <vfs_freevnodes_rollup>
0xffffffff80c60c2f <+143>: mov    $0x1,%eax
0xffffffff80c60c34 <+148>: pop    %rbp
0xffffffff80c60c35 <+149>: ret

```

End of assembler dump.

Σε αυτό το σημείο είναι ενδιαφέρον να τονιστεί ότι οι επιπλέον γραμμές με διευθύνσεις που τύπωσε το Lua script στην ενότητα 4.2.5, εμφανίζονται όλες στο παραπάνω disassembly. Προς υπενθύμιση, οι διευθύνσεις που τύπωσε το script είναι οι εξής:

sym_lo=	0xffffffff80c60ba0
sym_hi=	0xffffffff80c60c36
inl_lo=	0xffffffff80c60be1
inl_hi=	0xffffffff80c60c2f

Τα sym_lo και sym_hi αντιστοιχούν στα όρια της καλούσας συνάρτησης, δηλαδή της vget_prep(). Πράγματι, στο παραπάνω disassembly της vget_prep() βλέπουμε ότι οι διευθύνσεις 0xffffffff80c60ba0 και 0xffffffff80c60c36 αντιστοιχούν στην πρώτη και την τελευταία αντίστοιχα εντολή της συνάρτησης. Τα inl_lo και inl_hi αντιστοιχούν στα όρια του inline αντιγράφου της vfs_freevnodes_dec() που εντοπίστηκε εντός της vget_prep().

Τα entry και return offsets, λοιπόν, υπολογίζονται ως εξής:

$$Entry = Inline_{low} - Caller_{low}$$

$$Return = Inline_{high} - Caller_{low}$$

Χρησιμοποιώντας το παράδειγμα της ενότητας 4.2.5, μπορούμε να υπολογίσουμε τα offsets:

$$\begin{aligned} Entry &= Inline_{low} - Caller_{low} = 0xffffffff80c60be1 - 0xffffffff80c60ba0 = 65 \\ Return &= Inline_{high} - Caller_{low} = 0xffffffff80c60c2f - 0xffffffff80c60ba0 = 143 \end{aligned} \quad (4)$$

Ωστόσο, ύστερα από αρκετά πειράματα και ανάγνωση των disassembly διαφόρων συναρτήσεων, διαπιστώθηκε ότι υπάρχουν προβλήματα που περιπλέκουν τους υπολογισμούς των *Return* offsets, και η διαδικασία δεν είναι πάντα τόσο απλή, και το *Return* offset που μόλις υπολογίστηκε στην εξίσωση 4, στην πραγματικότητα δεν είναι σωστό.

- Το DIE inline αντιγράφου ορίζει τα όρια του με τις ιδιότητες *DW_AT_lowpc* και *DW_AT_highpc*, τότε η διεύθυνση επιστροφής του άνω ορίου είναι πάντα εκτός των ορίων του inline αντιγράφου.
- Το DIE inline αντιγράφου ορίζει τα όρια του με την ιδιότητα *DW_AT_ranges*, στην οποία περίπτωση μόνο το άνω όριο του τελευταίου range είναι εκτός των ορίων του inline αντιγράφου.
- Με βάση τα δύο παραπάνω σενάρια, αν το άνω όριο του inline αντιγράφου είναι το ίδιο με το άνω όριο της καλούσας συνάρτησης, τότε η διεύθυνση είναι εκτός και του inline αντιγράφου αλλά και της καλούσας συνάρτησης.

Αυτό είναι αποτέλεσμα της ίδιας της κωδικοποίησης DWARF. Σε αυτές τις 3 περιπτώσεις, μεταφέρουμε το άνω όριο του inline αντιγράφου μία εντολή πίσω. Οπότε, στην προκειμένη περίπτωση, το άνω όριο που βρέθηκε στην εξίσωση 1 και χρησιμοποιήθηκε μέχρι και τώρα ως παράδειγμα, στην πραγματικότητα πρέπει να μεταφερθεί μία εντολή πίσω. Σε όλες τις υπόλοιπες περιπτώσεις, το *Return* offset μπορεί να υπολογιστεί όπως παρουσιάστηκε αρχικά.

Οπότε, βάσει του disassembly της *vget_prep()*, το άνω όριο του inline αντιγράφου πρέπει να μεταφερθεί στην διεύθυνση *0xffffffff80c60c2a*, οπότε και θα καταλήξουμε με ένα νέο offset, το 138, αντί για το 143:

```
0xffffffff80c60c2a <+138>:   call    0xffffffff80c6a960 <vfs_freevnodes_rollup>
```

Μπορούμε να επαληθεύσουμε περαιτέρω ότι αυτό είναι έγκυρο, κοιτώντας στην υλοποίηση της *vfs_freevnodes_dec()*:

```
static __inline void
vfs_freevnodes_dec(void)
{
    int8_t *lfreevnodes;

    critical_enter();
    lfreevnodes = PCPU_PTR(vfs_freevnodes);
    (*lfreevnodes)--;
    if (__predict_false(*lfreevnodes == -VNLRU_FREEVNODES_SLOP))
        vfs_freevnodes_rollup(lfreevnodes);
    else
        critical_exit();
}
```

Στο offset που υπολογίσαμε ως άνω όριο του inline αντιγράφου της *vfs_freevnodes_dec()* εκτελείται η εντολή assembly *call vfs_freevnodes_rollup*, η οποία πράγματι είναι και η τελευταία εντολή του παραπάνω κομματιού κώδικα.

Επομένως, επιστρέφοντας στους υπολογισμούς, τα *entry* και *return* offsets του inline αντιγράφου εντός της *vget_prep()*, καταλήγουμε ότι είναι τα εξής:

$$\begin{aligned} \text{Entry} &= \text{Inline}_{\text{low}} - \text{Caller}_{\text{low}} = 0xffffffff80c60be1 - 0xffffffff80c60ba0 = 65 \\ \text{Return} &= \text{Inline}_{\text{high}} - \text{Caller}_{\text{low}} = 0xffffffff80c60c2a - 0xffffffff80c60ba0 = 138 \end{aligned} \quad (5)$$

Τέλος, τα offsets που υπολογίσθηκαν για κάθε inline αντίγραφο της `vfs_freevnodes_dec()`, χρησιμοποιούνται για την δημιουργία κανονικών `kinst` probes της μορφής `kinst::<function>:<instruction>`. Θα χρησιμοποιήσουμε και την επιλογή `-d` στην εντολή `dtrace(1)`, ώστε να εμφανισθεί το D script μετά τους συντακτικούς μετασχηματισμούς της `libdtrace`, δηλαδή της δημιουργίας των `kinst` probes.

Αρχικά, το `entry` probe, το οποίο θα δημιουργήσει `kinst` probes για την αρχή κάθε inline αντιγράφου της `vfs_freevnodes_dec()`:

```
# dtrace -dn 'kinst::vfs_freevnodes_dec:entry'
kinst::vnlru_free_impl:190,
kinst::bgetvp:203,
kinst::vhold:21,
kinst::vget_prep_smr:98,
kinst::vhold_smr:44,
kinst::vget_prep:65,
kinst::vflush:243,
kinst::mnt_vnode_next_lazy:257,
kinst::vop_rename_pre:33,
kinst::vop_rename_pre:127,
kinst::vop_rename_pre:215,
kinst::vop_rename_pre:300,
kinst::vfs_kqfilter:131,
kinst::vnlru_proc:2244,
kinst::sched_sync:798
{
}

kinst_enable:252: kinst: This provider is experimental, exercise caution
dtrace: description 'kinst::vfs_freevnodes_dec:entry' matched 15 probes
CPU      ID                FUNCTION:NAME
  1   85174                vget_prep_smr:98
  1   85174                vget_prep_smr:98
  5   85176                vget_prep:65
  4   85174                vget_prep_smr:98
  4   85174                vget_prep_smr:98
  4   85176                vget_prep:65
  5   85174                vget_prep_smr:98
  5   85174                vget_prep_smr:98
[...]
```

Αντίστοιχα, και το `return` probe, το οποίο θα δημιουργήσει `kinst` probes για όλα τα σημεία επιστροφής του κάθε inline αντιγράφου της `vfs_freevnodes_dec()`:

```
# dtrace -dn 'kinst::vfs_freevnodes_dec:return'
kinst::vnlru_free_impl:256,
kinst::vnlru_free_impl:769,
kinst::vnlru_free_impl:1110,
kinst::bgetvp:260,
```

```

kinst::bgetvp:400,
kinst::vhold:80,
kinst::vhold:96,
kinst::vget_prep_smr:164,
kinst::vget_prep_smr:175,
kinst::vhold_smr:103,
kinst::vhold_smr:138,
kinst::vget_prep:138,
kinst::vflush:310,
kinst::vflush:1146,
kinst::mnt_vnode_next_lazy:324,
kinst::mnt_vnode_next_lazy:789,
kinst::vop_rename_pre:100,
kinst::vop_rename_pre:417,
kinst::vop_rename_pre:457,
kinst::vop_rename_pre:194,
kinst::vop_rename_pre:436,
kinst::vop_rename_pre:473,
kinst::vop_rename_pre:274,
kinst::vop_rename_pre:388,
kinst::vop_rename_pre:359,
kinst::vop_rename_pre:454,
kinst::vop_rename_pre:492,
kinst::vfs_kqfilter:190,
kinst::vfs_kqfilter:247,
kinst::vnlru_proc:2304,
kinst::vnlru_proc:2836,
kinst::vnlru_proc:2853,
kinst::sched_sync:855,
kinst::sched_sync:1758,
kinst::sched_sync:1779
{
}

```

dtrace: description 'kinst::vfs_freevnodes_dec:return' matched 35 probes

CPU	ID	FUNCTION:NAME
1	85203	sched_sync:855
4	85203	sched_sync:855
4	85203	sched_sync:855
4	85203	sched_sync:855
0	85176	vhold:80
1	85203	sched_sync:855

Απόδειξη ότι η υλοποίηση πράγματι λειτουργεί, είναι ότι και στα δύο παραδείγματα παρατηρούμε ότι δημιουργήθηκαν probes για το inline αντίγραφο που εξετάζαμε ως τώρα, δηλαδή αυτό εντός της `vget_prep()`. Τα offsets για αυτό το inline αντίγραφο παρουσιάστηκαν στην εξίσωση 5 (*Entry* = 65 και *Return* = 138), και πράγματι βλέπουμε στο παράδειγμα της εκτέλεσης του probe `kinst::vfs_freevnodes_dec:entry` ότι ένα από τα probes που δημιουργήθηκαν ήταν το `kinst::vget_prep:65`, ενώ για το `kinst::vfs_freevnodes_dec:return` δημιουργήθηκε το `kinst::vget_prep:138`.

4.3 kinst tests

Ο κώδικας με τα Kyua test scripts του kinst βρίσκεται στο `/usr/src/cddl/contrib/opensolaris/cmd/dtrace/test/tst/common/kinst` [11]:

```
/usr/src/cddl/contrib/opensolaris/cmd/dtrace/test/tst/common/kinst
├── tst.basic.ksh
├── tst.dtracetest_fbtconvert.ksh
├── tst.dtracetest_inline.ksh
├── tst.inline_agg.ksh
└── tst.inline_multiple.ksh
```

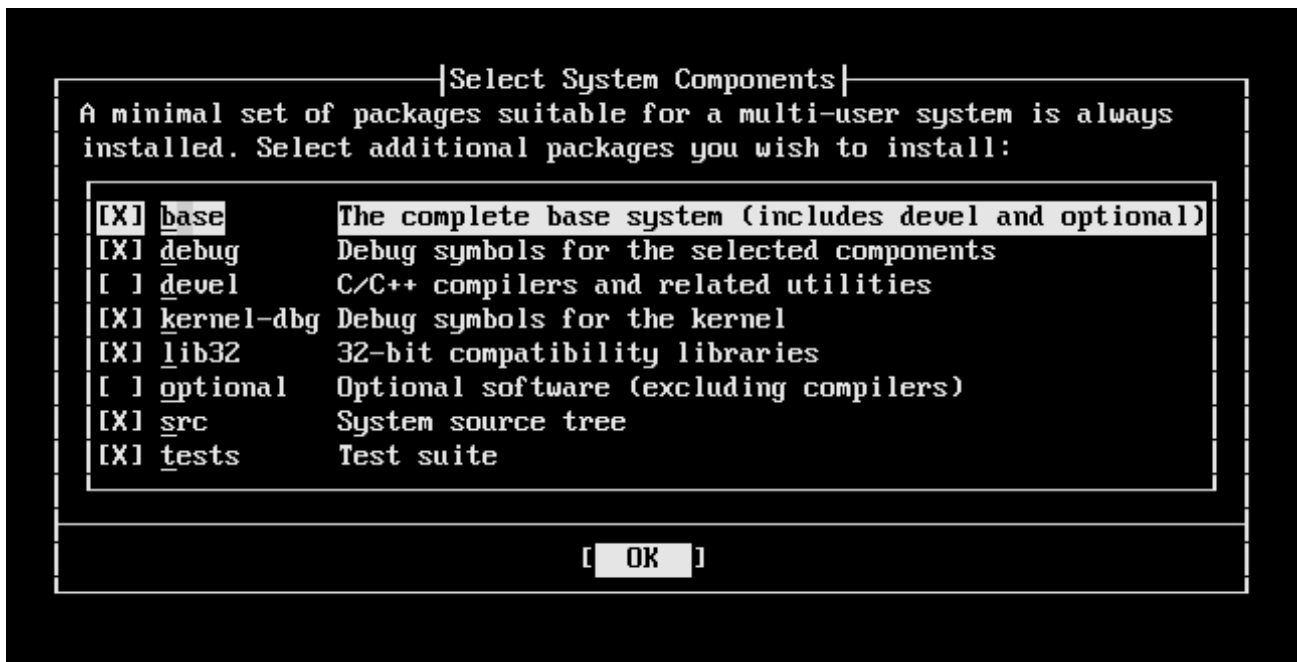
5 Πειράματα

Σε αυτό το κεφάλαιο θα παρουσιαστούν δοκιμές του `kinst` σε διάφορα σενάρια.

5.1 Περιβάλλον εκτέλεσης

Τα περιβάλλοντα στα οποία έγιναν οι δοκιμές είναι εικονικές μηχανές εντός του προσωπικού φορητού υπολογιστή του συγγραφέα της εργασίας. Χρησιμοποιήθηκαν τρεις εικονικές μηχανές, μία για κάθε αρχιτεκτονική που υποστήριζει ο `kinst` (AMD64, ARM64, RISC-V).

Κατά τις εγκαταστάσεις του λειτουργικού συστήματος στις εικονικές μηχανές, πρέπει επίσης να επιλέξουμε την εγκατάσταση του πηγαιού κώδικα (`src`), test suite (`tests`), καθώς και των debug αρχείων του userspace και του kernel (`debug` και `kernel-dbg` αντίστοιχα):



Παρόλο που δεν αφορά το κομμάτι της εργασίας, πληροφοριακά, τα χαρακτηριστικά του φορητού υπολογιστή στον οποίον τρέχουν οι εικονικές μηχανές είναι τα εξής:

- Λειτουργικό σύστημα: FreeBSD 14.3-RELEASE
- Μοντέλο υπολογιστή: Framework 13
- Επεξεργαστής: AMD Ryzen™ 7 7840U @3.3GHz (8 cores, 16 threads)
- Μνήμη RAM: 48GB

5.1.1 AMD64

- Λειτουργικό σύστημα: FreeBSD 15.0-ALPHA5
- Περιβάλλον εικονικοποίησης: bhyve
- Επεξεργαστής: 8 vCPU
- Μνήμη RAM: 8GB

Λήψη του πιο πρόσφατου snapshot του FreeBSD 15.0-ALPHA5:

```
# fetch https://download.freebsd.org/snapshots/amd64/amd64/ISO-IMAGES/15.0/FreeBSD-15.0-ALPHA5-amd64-20251004-1c0898edf28f-280541-disc1.iso
```

Δημιουργία άδειου αρχείου 8GB, που θα χρησιμοποιηθεί ως εικονικός σκληρός δίσκος για την εγκατάσταση του λειτουργικού συστήματος:

```
# truncate -s 8G /tmp/amddisk.img
```

Έναρξη εικονικής μηχανής:

```
# /usr/share/examples/bhyve/vmrun.sh \  
-AEG 1234 \  
-c 8 \  
-m 8G \  
-I ~/dl/FreeBSD-15.0-ALPHA5-amd64-20251004-1c0898edf28f-280541-disc1.iso \  
-d /tmp/amddisk.img \  
fbstdvm
```

Αφού εγκαταστήσουμε το λειτουργικό σύστημα και κάνουμε επανεκκίνηση, από εδώ και πέρα ξεκινάμε την εικονική μηχανή με την ίδια εντολή, αλλά χωρίς την γραμμή `-I . . .`, ώστε η εκκίνηση να γίνεται από τον εικονικό δίσκο που εγκαταστάθηκε το λειτουργικό σύστημα, και όχι από το αρχείο εγκατάστασης.

5.1.2 ARM64

- Λειτουργικό σύστημα: FreeBSD 15.0-ALPHA5
- Περιβάλλον εικονικοποίησης: QEMU
- Επεξεργαστής: 8 vCPU
- Μνήμη RAM: 8GB

Λήψη του πιο πρόσφατου snapshot του FreeBSD 15.0-ALPHA5:

```
# fetch https://download.freebsd.org/snapshots/arm64/aarch64/ISO-IMAGES/15.0/FreeBSD-15.0-ALPHA5-arm64-aarch64-20251004-1c0898edf28f-280541-disc1.iso
```

Εγκατάσταση απαραίτητων πακέτων για την λειτουργία της εικονικής μηχανής:

```
# pkg install u-boot-qemu-arm64
```

Δημιουργία άδειου αρχείου 8GB, που θα χρησιμοποιηθεί ως εικονικός σκληρός δίσκος για την εγκατάσταση του λειτουργικού συστήματος:

```
# truncate -s 8G /tmp/armdisk.img
```

Έναρξη εικονικής μηχανής:

```
# qemu-system-aarch64 \  
-s \  
-machine virt \  
-cpu cortex-a57 \  
-nographic \  
-smp 8 \  
-m 8G \  
-bios /usr/local/share/u-boot/u-boot-qemu-arm64/u-boot.bin \  

```



```
-cdrom ~/dl/FreeBSD-15.0-ALPHA5-arm64-aarch64-20251004-1c0898edf28f-280541-disc1.iso \  
-drive file=/tmp/armdisk.img,format=raw,id=hd0,if=none,media=disk \  
-device virtio-blk-device,drive=hd0 \  
-netdev user,hostfwd=tcp::10000-:22,id=net0 \  
-device virtio-net-device,netdev=net0
```

Αφού εγκαταστήσουμε το λειτουργικό σύστημα και κάνουμε επανεκκίνηση, από εδώ και πέρα ξεκινάμε την εικονική μηχανή με την ίδια εντολή, αλλά χωρίς την γραμμή `-cdrom` . . ., ώστε η εκκίνηση να γίνεται από τον εικονικό δίσκο που εγκαταστάθηκε το λειτουργικό σύστημα, και όχι από το αρχείο εγκατάστασης.

5.1.3 RISC-V

- Λειτουργικό σύστημα: FreeBSD 15.0-ALPHA5
- Περιβάλλον εικονικοποίησης: QEMU
- Επεξεργαστής: 8 vCPU
- Μνήμη RAM: 8GB

Λήψη του πιο πρόσφατου snapshot του FreeBSD 15.0-ALPHA5:

```
# fetch https://download.freebsd.org/snapshots/riscv/riscv64/ISO-IMAGES/15.0/FreeBSD-15.0-ALPHA5-riscv-riscv64-20251004-1c0898edf28f-280541-disc1.iso
```

Εγκατάσταση απαραίτητων πακέτων για την λειτουργία της εικονικής μηχανής:

```
# pkg install opensbi u-boot-qemu-riscv64
```

Δημιουργία άδειου αρχείου 8GB, που θα χρησιμοποιηθεί ως εικονικός σκληρός δίσκος για την εγκατάσταση του λειτουργικού συστήματος:

```
# truncate -s 8G /tmp/riscvdisk.img
```

Έναρξη εικονικής μηχανής:

```
# qemu-system-riscv64 \  
-s \  
-machine virt \  
-nographic \  
-smp 8 \  
-m 8G \  
-bios /usr/local/share/opensbi/lp64/generic/firmware/fw_jump.elf \  
-kernel /usr/local/share/u-boot/u-boot-qemu-riscv64/u-boot.bin \  
-cdrom ~/dl/FreeBSD-15.0-ALPHA5-riscv-riscv64-20251004-1c0898edf28f-280541-disc1.iso \  
-drive file=/tmp/riscvdisk.img,format=raw,id=hd0,if=none,media=disk \  
-device virtio-blk-device,drive=hd0 \  
-netdev user,hostfwd=tcp::10000-:22,id=net0 \  
-device virtio-net-device,netdev=net0
```

Αφού εγκαταστήσουμε το λειτουργικό σύστημα και κάνουμε επανεκκίνηση, από εδώ και πέρα ξεκινάμε την εικονική μηχανή με την ίδια εντολή, αλλά χωρίς την γραμμή `-cdrom` . . ., ώστε η εκκίνηση να γίνεται από τον εικονικό δίσκο που εγκαταστάθηκε το λειτουργικό σύστημα, και όχι από το αρχείο εγκατάστασης.

5.2 Μεταγλώττιση DTrace

Ο `kinst` περιλαμβάνεται στην κανονική έκδοση του DTrace από το FreeBSD 14.0 και έπειτα, όμως παρακάτω παρουσιάζεται η χειροκίνητη εγκατάσταση του kernel μέρους του DTrace — στο οποίο περιλαμβάνεται και ο κώδικας του `kinst` — για εγκυκλοπαιδικούς σκοπούς:

```
# cd /usr/src/sys/modules/dtrace
# make all install DEBUG_FLAGS=-g KMODDIR=/boot/kernel
```

Ο κώδικας της παρακολούθησης inline συναρτήσεων, εφόσον βρίσκεται ακόμα σε κατάσταση επιθεώρησης, δεν είναι μέρος του FreeBSD, οπότε πρέπει να τον κατεβάσουμε από το <https://reviews.freebsd.org/D38825>, να τον τοποθετήσουμε στον πηγαίο κώδικα του FreeBSD, και να τον μεταγλωττίσουμε χειροκίνητα:

```
# cd /usr/src
# fetch https://reviews.freebsd.org/D38825?download=true -o - 2>/dev/null | patch -p1
# cd cddl/lib/libdtrace
# make -j8 all install DEBUG_FLAGS=-g
```

5.3 Δοκιμές

Φόρτωση του `kinst` kernel module:

```
# kldload kinst
```

5.3.1 AMD64

Παρακολούθηση όλων των εντολών της συνάρτησης `vm_fault()`:

```
root@amd:~ # dtrace -n 'kinst::vm_fault:'
dtrace: description 'kinst::vm_fault:' matched 1334 probes
CPU      ID          FUNCTION:NAME
  2       4          vm_fault:0
  2       5          vm_fault:1
  2       6          vm_fault:4
  2       7          vm_fault:6
  2       8          vm_fault:8
  2       9          vm_fault:10
  2      10          vm_fault:12
  2      11          vm_fault:13
  2      12          vm_fault:20
  2      13          vm_fault:27
  2      14          vm_fault:34
  2      15          vm_fault:39
  2      16          vm_fault:48
[...]
```

Παρακολούθηση όλων των εντολών της συνάρτησης `spinlock_enter()` και εκτύπωση της στοίβας κλήσεων:

```
root@amd:~ # dtrace -n 'kinst::spinlock_enter: { stack(); }'
dtrace: description 'kinst::spinlock_enter: ' matched 18 probes
CPU      ID          FUNCTION:NAME
  1       4          spinlock_enter:0
                kernel'smp_rendezvous_cpus+0x92
```

```

dtrace.ko'dtrace_xcall+0xed
dtrace.ko'dtrace_state_go+0xb5d
dtrace.ko'dtrace_ioctl+0x10fa
kernel'devfs_ioctl+0xcb
kernel'vn_ioctl+0xc8
kernel'devfs_ioctl_f+0x1e
kernel'kern_ioctl+0x255
kernel'sys_ioctl+0x101
kernel'amd64_syscall+0x126
kernel'0xffffffff8105233b

```

```

1      5      spinlock_enter:1
0xffffffffe008c801960
dtrace.ko'dtrace_xcall+0xed
dtrace.ko'dtrace_state_go+0xb5d
dtrace.ko'dtrace_ioctl+0x10fa
kernel'devfs_ioctl+0xcb
kernel'vn_ioctl+0xc8
kernel'devfs_ioctl_f+0x1e
kernel'kern_ioctl+0x255
kernel'sys_ioctl+0x101
kernel'amd64_syscall+0x126
kernel'0xffffffff8105233b

```

```

2      5      spinlock_enter:1
0xffffffffe008c428dd0
kernel'cpu_idle+0xa6
kernel'sched_idletd+0x546
kernel'fork_exit+0x7b
kernel'0xffffffff81052a6e

```

[...]

Παρακολούθηση της πρώτης εντολής της συνάρτησης `kern_ioctl()` και εκτύπωση του καταχωρητή SP (stack pointer). Τα ονόματα των καταχωρητών του AMD64 που μπορούν να τυπωθούν ορίζονται στο αρχείο `/usr/lib/dtrace/regs_x86.d`:

```

root@amd:~ # dtrace -n 'kinst::kern_ioctl:0 { printf("%#x", regs[R_SP]); }'

```

```

dtrace: description 'kinst::kern_ioctl:0 ' matched 1 probe

```

CPU	ID	FUNCTION:NAME
6	4	kern_ioctl:0 0xffffffffe008ca77d48
6	4	kern_ioctl:0 0xffffffffe008ca77d48
6	4	kern_ioctl:0 0xffffffffe008ca77d48
6	4	kern_ioctl:0 0xffffffffe008ca77d48

[...]

Παρακολούθηση της αρχής της *inline* συνάρτησης `critical_enter()` και εκτύπωση του ονόματος του thread από το οποίο εκτελείται το probe, καθώς και της στοίβας κλήσης:

```

root@amd:~ # dtrace -n 'kinst::critical_enter:entry { printf("%s", curthread->td_name);
stack(); }'

```

```

dtrace: description 'kinst::critical_enter:entry ' matched 183 probes

```

CPU	ID	FUNCTION:NAME
-----	----	---------------

```

2      158      spinlock_enter:53 dtrace
0xffffffff008caef960
kernel'smp_rendezvous_cpus+0x92
dtrace.ko'dtrace_xcall+0xed
dtrace.ko'dtrace_state_go+0xb5d
dtrace.ko'dtrace_ioctl+0x10fa
kernel'devfs_ioctl+0xcb
kernel'vn_ioctl+0xc8
kernel'devfs_ioctl_f+0x1e
kernel'kern_ioctl+0x255
kernel'sys_ioctl+0x101
kernel'amd64_syscall+0x126
kernel'0xffffffff8105233b

6      158      spinlock_enter:53 dbuf_evict_thread
0xffffffff008c761e00
kernel'sleepq_lock+0x21
kernel'_cv_timedwait_sbt+0x73
zfs.ko'dbuf_evict_thread+0x385
kernel'fork_exit+0x7b
kernel'0xffffffff81052a6e

1      158      spinlock_enter:53 swi0: uart uart++
0xffffffff008c635be0
kernel'ns8250_bus_ipend+0x21
kernel'uart_intr+0x80
kernel'intr_event_handle+0xc4
kernel'intr_execute_handlers+0x49
kernel'0xffffffff81052d4c
kernel'uart_tty_intr+0x242
kernel'ithread_loop+0x239
kernel'fork_exit+0x7b
kernel'0xffffffff81052a6e

[...]
```

Στην δεύτερη γραμμή παρατηρούμε ότι δημιουργήθηκαν 183 probes, το οποίο σημαίνει ότι βρέθηκαν 183 inline αντίγραφα της αρχής της συνάρτησης `critical_enter()`.

5.3.2 ARM64

Παρακολούθηση όλων των εντολών της συνάρτησης `vm_fault()`:

```

root@arm:~ # dtrace -n 'kinst::vm_fault:'
dtrace: description 'kinst::vm_fault:' matched 1442 probes
CPU      ID      FUNCTION:NAME
  6       4      vm_fault:0
  6       5      vm_fault:4
  6       6      vm_fault:8
  6       7      vm_fault:12
  6       8      vm_fault:16
  6       9      vm_fault:20
```

```

6      10      vm_fault:24
6      11      vm_fault:28
6      12      vm_fault:32
6      13      vm_fault:44
6      14      vm_fault:48
6      15      vm_fault:52
6      16      vm_fault:56
6      17      vm_fault:60
[...]
```

Παρακολούθηση όλων των εντολών της συνάρτησης `spinlock_enter()` και εκτύπωση της στοίβας κλήσεων:

```

root@arm:~ # dtrace -n 'kinst::spinlock_enter: { stack(); }'
dtrace: description 'kinst::spinlock_enter: ' matched 22 probes
CPU      ID      FUNCTION:NAME
  4       4      spinlock_enter:0
              kernel'handle_el1h_sync+0x18
              0xffff0000c08ea32c

  4       5      spinlock_enter:4
              kernel'handle_el1h_sync+0x18
              0xffff0000c08ea32c

  4       6      spinlock_enter:8
              kernel'handle_el1h_sync+0x18
              0xffff0000c08ea39c

  4       7      spinlock_enter:12
              kernel'handle_el1h_sync+0x18
              0xffff0000c08ea39c

  4       8      spinlock_enter:16
              kernel'handle_el1h_sync+0x18
              0xffff0000c08ea39c
[...]
```

Παρακολούθηση της πρώτης εντολής της συνάρτησης `kern_ioctl()` και εκτύπωση του καταχωρητή SP (stack pointer). Τα ονόματα των καταχωρητών του ARM64 που μπορούν να τυπωθούν ορίζονται στο αρχείο `/usr/lib/dtrace/regs_aarch64.d`:

```

root@arm:~ # dtrace -n 'kinst::kern_ioctl:0 { printf("%#x", regs[R_SP]); }'
dtrace: description 'kinst::kern_ioctl:0 ' matched 1 probe
CPU      ID      FUNCTION:NAME
  0    65058      kern_ioctl:0 0xffff0000c0854750
  0    65058      kern_ioctl:0 0xffff0000c0854750
  0    65058      kern_ioctl:0 0xffff0000c0854750
[...]
```

Παρακολούθηση της αρχής της *inline* συνάρτησης `critical_enter()` και εκτύπωση του ονόματος του thread από το οποίο εκτελείται το probe, καθώς και της στοίβας κλήσης:

```

root@arm:~ # dtrace -n 'kinst::critical_enter:entry { printf("%s", curthread->td_name);
stack(); }'
```

```

dtrace: description 'kinst::critical_enter:entry ' matched 157 probes
CPU      ID                      FUNCTION:NAME
  1  65191                      spinlock_enter:60 dtrace
                        kernel'handle_el1h_sync+0x18
                        kernel'arm_gic_intr+0x118
                        kernel'intr_irq_handler+0x84
                        kernel'_thread_lock+0x24
                        kernel'sigallowstop_impl+0x4c
                        kernel'ttydev_write+0x26c
                        kernel'devfs_write_f+0xf0
                        kernel'dofilewrite+0x7c
                        kernel'sys_write+0xf4
                        kernel'do_el0_sync+0x6b4
                        kernel'handle_el0_sync+0x4c

  6  65190                      cpu_idle:172 idle: cpu6
                        kernel'handle_el1h_sync+0x18
                        kernel'cpu_activeclock+0x78
                        kernel'cpu_idle+0x8c

  5  65100                      tc_cpu_ticks:20 bufspacedaemon-0
                        kernel'handle_el1h_sync+0x18
                        kernel'callout_lock+0x68
                        kernel'callout_reset_sbt_on+0x54
                        kernel'mi_switch+0x70
                        kernel'sleepq_timedwait+0x2c
                        kernel'_sleep+0x1b4
                        kernel'bufspace_daemon+0x4f8
                        kernel'fork_exit+0x84
                        kernel'fork_trampoline+0x18

  1  65187                      intr_irq_handler:100 swi0: uart
                        kernel'handle_el1h_sync+0x18
                        0xffff000041a18604

[...]
```

Στην δεύτερη γραμμή παρατηρούμε ότι δημιουργήθηκαν 157 probes, το οποίο σημαίνει ότι βρέθηκαν 157 inline αντίγραφα της αρχής της συνάρτησης `critical_enter()`.

5.3.3 RISC-V

Παρακολούθηση όλων των εντολών της συνάρτησης `vm_fault()`:

```

root@riscv:~ # dtrace -n 'kinst::vm_fault:'
dtrace: description 'kinst::vm_fault:' matched 1658 probes
CPU      ID                      FUNCTION:NAME
  2  47548                      vm_fault:0
  2  47549                      vm_fault:2
  2  47550                      vm_fault:4
  2  47551                      vm_fault:6
  2  47552                      vm_fault:8
```

```

2 47553                vm_fault:10
2 47554                vm_fault:12
2 47555                vm_fault:14
2 47556                vm_fault:16
2 47557                vm_fault:18
2 47558                vm_fault:20
2 47559                vm_fault:22
2 47560                vm_fault:24
2 47561                vm_fault:26
[...]
```

Παρακολούθηση όλων των εντολών της συνάρτησης `spinlock_enter()` και εκτύπωση της στοίβας κλήσεων:

```

root@riscv:~ # dtrace -n 'kinst::spinlock_enter: { stack(); }'
dtrace: description 'kinst::spinlock_enter: ' matched 22 probes
 2 47554                spinlock_enter:16
                        kernel'spinlock_enter+0x10
                        kernel'do_trap_supervisor+0x78
                        kernel'cpu_exception_handler_supervisor+0x74
                        kernel'cpu_idleclock+0xc8
                        kernel'cpu_idle+0x22
                        kernel'sched_idletd+0x456
                        kernel'fork_exit+0x66

4 47558                spinlock_enter:30
                        kernel'spinlock_enter+0x1e
                        kernel'do_trap_supervisor+0x78
                        kernel'cpu_exception_handler_supervisor+0x74
                        kernel'handleevents+0x1b0
                        kernel'timercb+0x262
                        kernel'riscv_timer_intr+0x38
                        kernel'intr_event_handle+0x84
                        kernel'intr_isrc_dispatch+0x2a
                        kernel'intc_intr+0x3e
                        kernel'intr_irq_handler+0x52
                        kernel'do_trap_supervisor+0x5e
                        kernel'cpu_exception_handler_supervisor+0x74
                        kernel'smp_rendezvous_cpus+0x17e
                        dtrace.ko'dtrace_state_go+0x800
                        dtrace.ko'dtrace_ioctl+0x7a4
                        kernel'devfs_ioctl+0xa6
                        kernel'vn_ioctl+0x1c2
                        kernel'devfs_ioctl_f+0x1e
                        kernel'kern_ioctl+0x28a
                        kernel'sys_ioctl+0xea

2 47555                spinlock_enter:18
                        kernel'spinlock_enter+0x12
                        kernel'do_trap_supervisor+0x78
                        kernel'cpu_exception_handler_supervisor+0x74
                        kernel'cpu_idleclock+0xc8
```

```
kernel'cpu_idle+0x22
kernel'sched_idletd+0x456
kernel'fork_exit+0x66
```

[...]

Παρακολούθηση της πρώτης εντολής της συνάρτησης `kern_ioctl()` και εκτύπωση του καταχωρητή SP (stack pointer). Τα ονόματα των καταχωρητών του RISC-V που μπορούν να τυπωθούν ορίζονται στο αρχείο `/usr/lib/dtrace/regs_riscv.d`:

```
root@riscv:~ # dtrace -n 'kinst::kern_ioctl:0 { printf("%#x", regs[R_SP]); }'
dtrace: description 'kinst::kern_ioctl:0 ' matched 1 probe
CPU      ID                      FUNCTION:NAME
  0  47548                      kern_ioctl:0 0xffffffffc0826c6aa0
  0  47548                      kern_ioctl:0 0xffffffffc0826c6aa0
  0  47548                      kern_ioctl:0 0xffffffffc0826c6aa0
[...]
```

Παρακολούθηση της αρχής της *inline* συνάρτησης `critical_enter()` και εκτύπωση του ονόματος του thread από το οποίο εκτελείται το probe, καθώς και της στοίβας κλήσης:

```
root@riscv:~ # dtrace -n 'kinst::critical_enter:entry { printf("%s", curthread->td_name); stack(); }'
dtrace: description 'kinst::critical_enter:entry ' matched 139 probes
CPU      ID                      FUNCTION:NAME
  1  47570                      malloc:94 dtrace
                        kernel'malloc+0x5e
                        kernel'do_trap_supervisor+0x78
                        kernel'cpu_exception_handler_supervisor+0x74
                        dtrace.ko'dtrace_dof_create+0x1e
                        dtrace.ko'dtrace_ioctl+0xc42
                        kernel'devfs_ioctl+0xa6
                        kernel'vn_ioctl+0x1c2
                        kernel'devfs_ioctl_f+0x1e
                        kernel'kern_ioctl+0x28a
                        kernel'sys_ioctl+0xea
                        kernel'do_trap_user+0x1c6

  2  47678                      spinlock_enter:42 l2arc_feed_thread
                        kernel'spinlock_enter+0x2a
                        kernel'do_trap_supervisor+0x78
                        kernel'cpu_exception_handler_supervisor+0x74
                        kernel'handleevents+0x1b0
                        kernel'timercb+0x262
                        kernel'riscv_timer_intr+0x38
                        kernel'intr_event_handle+0x84
                        kernel'intr_isrc_dispatch+0x2a
                        kernel'intc_intr+0x3e
                        kernel'intr_irq_handler+0x52
                        kernel'do_trap_supervisor+0x5e
                        kernel'cpu_exception_handler_supervisor+0x74
                        kernel'mi_switch+0xf4
                        kernel'sleepq_switch+0xc8
```



```
kernel'sleepq_timedwait+0x24  
kernel'_cv_timedwait_sbt+0x108  
zfs.ko'l2arc_feed_thread+0x2fc  
kernel'fork_exit+0x66
```

[...]

Στην δεύτερη γραμμή παρατηρούμε ότι δημιουργήθηκαν 139 probes, το οποίο σημαίνει ότι βρέθηκαν 139 inline αντίγραφα της αρχής της συνάρτησης `critical_enter()`.

6 Συμπεράσματα και προτάσεις μελλοντικών εργασιών

Σε αυτή την διπλωματική εργασία μελετήθηκε το αντικείμενο της παρακολούθησης, ξεκινώντας από μία συνοπτική περιγραφή αυτής, στα πλαίσια του FreeBSD, και έπειτα έγινε αναλυτική αναφορά στο DTrace, πάνω στο οποίο αναπτύχθηκε ο νέος provider `kinst`, του οποίου επίσης έγινε χρήση στην `libdtrace`, με σκοπό την παρακολούθηση inline συναρτήσεων.

Ως μέρος της ανάπτυξης του `kinst` και της παρακολούθησης inline συναρτήσεων, μελετήθηκαν επίσης διάφορα κομμάτια που αφορούν την αρχιτεκτονική επεξεργαστών και τις γλώσσες `assembly` αυτών, τον πυρήνα του FreeBSD και την ανάπτυξη κώδικα πυρήνα, καθώς και πρότυπα αρχείων όπως το DWARF, CTF και ELF.

Πέρα από την περαιτέρω ανάπτυξή του και βελτίωση της αξιοπιστίας του σε περιβάλλοντα παραγωγής, μελλοντικά, μερικά από τα πράγματα που θα κάνουν τον `kinst` σημαντικά πιο εύχρηστο είναι η ανάπτυξη εργαλείων υψηλότερου επιπέδου. Ο `kinst` είναι ένας αρκετά low-level provider, και όπως είδαμε, στην απλή του χρήση (δηλαδή όχι παρακολούθηση inline συναρτήσεων) ο χρήστης χρειάζεται να γνωρίζει ακριβή offsets εντός των συναρτήσεων που παρακολουθεί. Η διαδικασία εύρεσης αυτών των offsets μέσω των `disassembly` των συναρτήσεων, σε πολλές περιπτώσεις μπορεί να είναι είτε χρονοβόρα είτε και αδύνατη, μίας και ο χρήστης πρέπει να διαβάζει κώδικα `assembly`, ο οποίος κώδικας της συνάρτησης αν αλλάξει κάποια στιγμή, αλλάζουν και τα offsets. Πολύ χρήσιμο θα ήταν ένα εργαλείο το οποίο μπορεί να δεχθεί ως είσοδο την συνάρτηση προς παρακολούθηση, καθώς και τον αριθμό της γραμμής κώδικα που θέλει να παρακολουθήσει, και να κάνει την μετάφραση από αριθμό γραμμής σε offset(s) αυτόματα.

Άλλα κομμάτια που αξίζουν να ερευνηθούν και να υλοποιηθούν εντός του `kinst`, είναι η παρακολούθηση σημείων επιστροφής `tail-call optimized` συναρτήσεων [26].

Τέλος, ο `kinst` θα ήταν πολύ χρήσιμο να υποστηρίζει την μεταβλητή `args[]`, όπως και ο `fbt`.

7 Βιβλιογραφία

- [1] Mark Johnston “Debugging the FreeBSD Kernel”
<https://freebsdoundation.org/wp-content/uploads/2019/01/Debugging-the-FreeBSD-Kernel.pdf>
- [2] Tamches, Ariel & Miller, Barton P. “Fine-Grained Dynamic Instrumentation of Commodity Operating System Kernels”
https://www.usenix.org/legacy/publications/library/proceedings/osdi99/full_papers/tamches/tamches.pdf
- [3] Illumos Operating System “Dynamic Tracing Guide”
<https://illumos.org/books/dtrace>
- [4] Oracle “Solaris Dynamic Tracing Guide”
https://docs.oracle.com/cd/E18752_01/html/817-6223/toc.html
- [5] Bryan M. Cantrill, Michael W. Shapiro, Adam H. Leventhal, “Dynamic Instrumentation of Production Systems”
https://www.usenix.org/legacy/event/usenix04/tech/general/full_papers/cantrill/cantrill.html/
- [6] Brendan Gregg “DTrace Topics: Introduction”
https://www.brendangregg.com/DTrace/dtrace_topics_intro.pdf
- [7] Arun Thomas “DTrace Internals: Digging into DTrace”
https://www.bsdcn.org/2017/schedule/attachments/432_dtrace_internals.pdf
- [8] Mark Johnston “Introduction to DTrace on FreeBSD”
<https://www.youtube.com/watch?v=E06GVdH-LX0>
- [9] FreeBSD src “kinst source code”
<https://cgit.freebsd.org/src/tree/sys/cddl/dev/kinst>
- [10] FreeBSD src “Inline tracing source code”
https://cgit.freebsd.org/src/tree/cddl/contrib/opensolaris/lib/libdtrace/common/dt_sugar.c
- [11] FreeBSD src “kinst tests source code”
<https://cgit.freebsd.org/src/tree/cddl/contrib/opensolaris/cmd/dtrace/test/tst/common/kinst>
- [12] FreeBSD Code Reviews “libdtrace: Implement inline function tracing for kinst”
<https://reviews.freebsd.org/D38825>
- [13] The DWARF Standard
<https://dwarfstd.org/>
- [14] Christos Margiolis “Using DWARF to find call sites of inline functions”
https://margiolis.net/en/w/dwarf_inline/
- [15] Christos Margiolis “Inline function tracing with the kinst DTrace provider”
https://margiolis.net/en/w/kinst_inline/
- [16] Intel “Intel 64 and IA-32 Architectures Software Developer Manuals”
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [17] ARM “A64 Instruction Set Architecture Guide”
<https://developer.arm.com/documentation/102374/latest/>

- [18] RISC-V International “RISC-V Ratified Specifications”
<https://riscv.org/specifications/ratified/>
- [19] msyksphinz “RISC-V Instruction Set Specifications”
<https://msyksphinz-self.github.io/riscv-isadoc/>
- [20] Brown University Department of Computer Science “Assembly Language, Calling Convention, and the Stack”
<https://cs.brown.edu/courses/csci1310/2020/notes/l08.html>
- [21] Wikipedia “Function prologue and epilogue”
https://en.wikipedia.org/wiki/Function_prologue_and_epilogue
- [22] Microsoft “The AArch64 processor (aka arm64), part 21: Classic function prologues and epilogues”
<https://devblogs.microsoft.com/oldnewthing/20220824-00/?p=107043>
- [23] Jim Crownie “Atomics in AArch64”
<https://cpufun.substack.com/p/atomics-in-aarch64>
- [24] TIS Committee “Executable and Linking Format (ELF) Specification”
<https://refspecs.linuxfoundation.org/elf/elf.pdf>
- [25] University of Wisconsin-Madison “Traps, interrupts, and drivers”
<https://pages.cs.wisc.edu/~gerald/cs537/Summer17/handouts/traps.pdf>
- [26] Wikipedia “Tail call”
https://en.wikipedia.org/wiki/Tail_call
- [27] GNU “GDB: The GNU Project Debugger”
<https://www.sourceware.org/gdb/>
- [28] FreeBSD manual pages “kgdb(1)”
[https://man.freebsd.org/cgi/man.cgi?query=kgdb\(1\)](https://man.freebsd.org/cgi/man.cgi?query=kgdb(1))
- [29] FreeBSD Manual Pages “ctf(5)”
<https://man.freebsd.org/cgi/man.cgi?query=ctf&sektion=5&format=html>

Συντομογραφίες

ABI	Application Binary Interface
CPU	Central Processing Unit
CTF	Compact C Type Format
DIE	Debugging Information Entry
DIF	D Intermediate Format
DWARF	Debugging with Attributed Record Formats
ELF	Executable and Linkable Format
FBT	Function Boundary Tracing
GDB	GNU Debugger
ISA	Instruction Set Architecture
PC	Program Counter
POSIX	Portable Operating System Interface
eBPF	Extended Berkley Packet Filter